



Ben-Gurion University of the Negev Faculty of Engineering Science

School of Electrical and Computer Engineering

Dept. of Electrical and Computer Engineering

Fourth Year Engineering Project

Final Report

Design and Implementation of a Hovering Autonomous Underwater Vehicle (HAUV)

Project Number:	p-2026-031
Students (Name and ID) :	Guy Hatzir - 207015272 Maayan Sarig - 206511776
Supervisor:	Prof. Hugo Guterman
Email for contact:	guyhatz@post.bgu.ac.il sarigma@post.bgu.ac.il
Submission Date:	26/07/2026

Table of Contents

1. Abstract	6
i. English Abstract	6
ii. Hebrew Abstract	7
2. Introduction	7
3. Final Spec Sheet	9
3.1 Project Components	9
3.1.1 USV Subsystems	9
3.1.2 ROV (Remotely Operated Vehicle) Subsystems	11
3.1.3 Communication & Networking Infrastructure	11
3.2 Key Engineering Changes Since PDR	13
3.2.1 Power Supply and PoE (Power over Ethernet) Integration	13
3.2.2 Network Routing, Static IPs, and Overcoming Restrictions	13
3.2.3 Implementation of the RTL (Return-to-Launch) Failsafe State	13
3.2.4 Three-Zone Kinematic Control Logic	14
3.2.5 GPS Injection and Relative Positioning Validation	15
3.2.6 Hardware-Timed PWM Generation (pigpio Daemon)	15
3.2.7 Mechanical and Algorithmic Overhaul of the Winch System	15
3.2.8 Thruster Polarity Reversal Delay (Preventing Motor Stalling)	15
4. Proposed Solution and Engineering Design Approach	17
4.1 General Problem-Solving Approach	17
4.2 Software Architecture and Block Diagram (ROS 2)	17
4.3 Flowchart - Tracking Logic and State Machine	19
4.3.1 Station Keeping	19
4.3.2 Manual	19
4.3.3 Auto	20
4.3.4 RTL	20
4.4 Electronic Circuit Design, Communication, and Hardware Timing	20
4.4.1 Central Computing & PWM Generation	20
4.4.2 Logic Conversion & Sensors	20
4.4.3 Network & PoE Delivery	21
4.5 Design Considerations	21
4.5.1 Master/Slave Architecture	21
4.5.2 ROV GPS Injection	21
4.5.3 Control Logic & Thresholds	22
4.5.4 RTL	22

5.	Final Product Acceptance Testing	23
5.1	Test Array and Measurement Methodology	23
5.1.1	Hardware-in-the-Loop (HIL) Verification.....	23
5.1.2	Autonomous Control Logic Validation	23
5.1.3	End-to-End System Integration.....	23
5.2	Measured Product Performance.....	23
5.2.1	USV Hardware Bring-up and Sensor Verification	23
5.2.2	Autonomous Tracking Logic Validation (Bench Testing)	24
5.2.3	Full System Integration and ROV Control	24
5.3	Conclusion of Acceptance Testing	25
5.4	List of Tests	26
6.	Problems and Solutions.....	28
6.1	Connecting and Configuring Wi-Fi on the Raspberry Pi 4	28
6.2	Remote Work and Hardware-Simulator Transition.....	28
6.3	IP Address Conflicts and Network Routing	28
6.4	RF Interference Between Communication Antennas	29
6.5	Power over Ethernet (PoE) Supply for Antennas.....	29
6.6	USV Motor Stalling During Sudden Direction Changes.....	29
6.7	ROV Restriction on Entering Autonomous 'Auto' Mode	29
7.	Conclusions and Recommendations	31
7.1	Project Success and Objective Achievement Analysis.....	31
7.2	Planned vs. Actual Comparison (Schedule & Budget).....	33
7.3	Discussion: Pros and Cons of the Proposed Architecture	34
7.3.1	System Advantages	34
7.3.2	System Disadvantages and Limitations	34
7.4	Recommendations for Future Work	35
7.4.1	Phase 1: Environmental Hardening and Power Autonomy	35
7.4.2	Phase 2: Sensor Augmentation and Mechanical Reliability	36
8	Bibliography	37
9	Appendix.....	38
9.1	Task Management	38
9.2	User Manual	38
9.3	Technical Manual	58
9.4	Links Table	106
9.5	Recommendation Sheet for Grading a Summary Report	107

Nomenclature Table

Acronym	Full Name
AC	Alternating Current
ADC	Analog-to-Digital Converter
BCM	Broadcom (GPIO pin-numbering scheme, as opposed to physical pin numbers)
BMS	Battery Management System
CG	Center of Gravity
COTS	Commercial Off-The-Shelf
CPE	Customer-Premises Equipment
DC	Direct Current
DIR	Direction (stepper-driver signal pin)
DOF	Degrees of Freedom
DT	Data (HX711 load-cell amplifier serial data-output pin)
DVL	Doppler Velocity Log
EKF	Extended Kalman Filter
ESC	Electronic Speed Controller
FXTI	Fathom-X Tether Interface
GND	Ground
GNGGA	NMEA sentence identifier for GNSS Fix Data (GN = multi-constellation talker ID, GGA = Fix Data)
GPIO	General-Purpose Input/Output
GPS	Global Positioning System
HAUV	Hovering Autonomous Underwater Vehicle
HDPE	High-Density Polyethylene
HTTP	HyperText Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
IMU	Inertial Measurement Unit
IP	Internet Protocol
LAN	Local Area Network
LB	Left Bumper (game-controller button)
LT	Left Trigger (game-controller button)
LTS	Long-Term Support
MAVLink	Micro Air Vehicle Link (telemetry/command protocol)
NMEA	National Marine Electronics Association
OS	Operating System
OVP	Over-Voltage Protection
PDR	Preliminary Design Review
PoE	Power over Ethernet
PWM	Pulse-Width Modulation

QGC	QGroundControl (ground control station software)
RB	Right Bumper (game-controller button)
RF	Radio Frequency
ROS	Robot Operating System
ROV	Remotely Operated Vehicle
RPi4	Raspberry Pi 4
RPM	Revolutions Per Minute
RT	Right Trigger (game-controller button)
RTL	Return-to-Launch
SCK	Serial Clock (HX711 load-cell amplifier clock pin)
SSH	Secure Shell
SSID	Service Set Identifier
TCP	Transmission Control Protocol
TTL	Transistor-Transistor Logic
UAV	Unmanned Aerial Vehicle
UBX	u-blox proprietary binary GPS protocol
UDP	User Datagram Protocol
UI	User Interface
USB	Universal Serial Bus
USV	Unmanned Surface Vehicle
UVL	Under-Voltage Lockout
VCC	Voltage Common Collector (supply-voltage pin)

Table 1: Table of Nomenclature

Note on system terminology: USV (Unmanned Surface Vehicle) refers only to the surface vehicle; ROV (Remotely Operated Vehicle / BlueROV2) refers only to the underwater vehicle; HAUV (Hovering Autonomous Underwater Vehicle) refers to the combined, integrated system of both platforms.

1. Abstract

i. English Abstract

Autonomous USV-Based Communication Relay for Extended-Range and Independent ROV Operations

Hatzir Guy, Sarig Maayan

guyhatz@post.bgu.ac.il sarigma@post.bgu.ac.il

Prof. Guterman Hugo

Remotely Operated Vehicles (ROVs) require stable two-way communication, but conventional RF (Radio Frequency) signals do not propagate underwater, necessitating surface relays.

This project aims to enable extended-range, autonomous ROV operations without a manned surface platform by developing an unmanned relay.

An Unmanned Surface Vehicle (USV) acts as an autonomous relay, connecting via long-range RF. It tracks the ROV using three mechanisms: a winch measuring tether length, an encoder measuring tether angle, and a tension meter. These enable autonomous movement coordination and maintain a stable communication link.

Dry-lab proof-of-concept testing validated autonomous execution, synchronized USV-ROV motor movements, and accurate integrated responses. Although pool and sea trials have not yet been conducted to verify extended-range performance, these bench-scale results demonstrate the system's foundational viability to eventually eliminate the need for manned support vessels.

Keywords: ROV, USV, Autonomous operation, RF, Remote Control, Underwater, Relay, Tracking.

ii. Hebrew Abstract

ממסר תקשורת אוטונומי מבוסס USV עבור פעולות ROV עצמאיות ובטווח רחב

חציר גיא, שריג מעין

guyhatz@post.bgu.ac.il sarigma@post.bgu.ac.il

פרופ. גוטרמן הוגו

רכבים תת-ימיים (ROV) דורשים תקשורת יציבה, אך גלי רדיו (RF) אינם עוברים במים, מה שמחייב שימוש בממסר על פני השטח.

מטרת הפרויקט היא לאפשר ל-ROV פעילות אוטונומית בטווח מוגדל, ללא צורך בפלטפורמה ימית מאוישת, באמצעות פיתוח ממסר תקשורת בלתי מאויש.

פותח כלי שיט בלתי מאויש (USV) כממסר אוטונומי מבוסס רדיו (RF) ארוך-טווח. ה-USV עוקב אחר ה-ROV בעזרת שלושה רכיבים: כננת למדידת אורך החוט, אנקודר לזווית החוט, ומד מתיחות. נתונים אלו מאפשרים סנכרון תנועה אוטונומי ושמירה על ערוץ תקשורת רציף.

בדיקות מעבדה אישרו ביצוע משימה אוטונומית, הפעלת רכיבים בפקודה, סנכרון מנועים מלא, וקריאות תקינות של אורך הכננת, זווית האנקודר ומד המתיחות.

רעיון ממסר ה-USV ומערכת המעקב האוטונומית מיייתרים ביעילות ספינות תמיכה מאוישות, ומרחיבים משמעותית את טווח הפעולה והאוטונומיה של רכבים תת-ימיים.

מילות מפתח: ROV, USV, הפעלה אוטונומית, תדרי רדיו, שליטה מרחוק, תת-ימי, ממסר, מעקב.

2. Introduction

The operational capabilities of Remotely Operated Vehicles (ROVs) are traditionally constrained by the necessity of a direct physical connection to a manned surface platform. Because conventional Radio Frequency (RF) communication cannot propagate underwater, a physical tether is essential for maintaining a reliable data link. This hardware limitation strictly dictates the operational range of the system and demands continuous, fatiguing manual piloting, effectively preventing large-scale, remote, or fully independent underwater operations.

To overcome these limitations, the primary objective of this project is to design a control and communication architecture that physically decouples the operator from the ROV's immediate

vicinity. Rather than relying on traditional surface support, the engineering goals focus on achieving reliable relative localization between surface and underwater entities without expensive sensor arrays, and ensuring robust, low-latency data routing over inherently unstable maritime wireless channels.

Recent advancements in heterogeneous marine robotic systems have extensively explored coupling USVs with ROVs to mitigate inherent underwater navigation drift and the complete absence of GPS signals. In such cooperative systems, the "Master - Slave" control paradigm and dynamic formation controls are predominantly employed to coordinate motion. While the foundational mathematical models for these dynamic formations are often adapted from highly agile aerial (UAV - Unmanned Aerial Vehicle) frameworks [2], these exact kinematic control principles transfer directly to the 6-DOF (Degrees of Freedom) open water environment, enabling the synchronized, multi-vessel operations increasingly utilized in modern USV architectures [3]. To achieve relative localization, previous marine studies - including foundational acoustic tracking research developed within our university group [1] - have heavily relied on acoustic 'Ranger' interrogator systems. However, rather than directly relying on this methodology, we utilized this prior work primarily as an inspiration to explore new directions for our system.

While acoustic positioning systems offer reliable relative tracking [1], they are prohibitively expensive, susceptible to acoustic noise, and highly complex to integrate. Instead, this project adopts a mechanical-kinematic approach, which utilizes the physical tether itself as a feedback mechanism. By actively monitoring the tether's tension and catenary angle, the control loop can effectively track the ROV, maintain an optimal separation distance, and prevent cable entanglement without exerting excessive drag. Additionally, to mitigate near-sea-surface RF channel [4], the system architecture specifically isolates low-latency MAVLink (Micro Air Vehicle Link) control telemetry from bandwidth-heavy video streams using a distributed ROS 2 (Robot Operating System) middleware on an onboard Raspberry Pi 4, aligning with modern modular communication infrastructures for maritime robotics [5]. This specific integration of tether-kinematics for localization and separated data-routing for communication provides a highly robust, cost-effective, and scalable solution for autonomous offshore operations.

3. Final Spec Sheet

3.1 Project Components

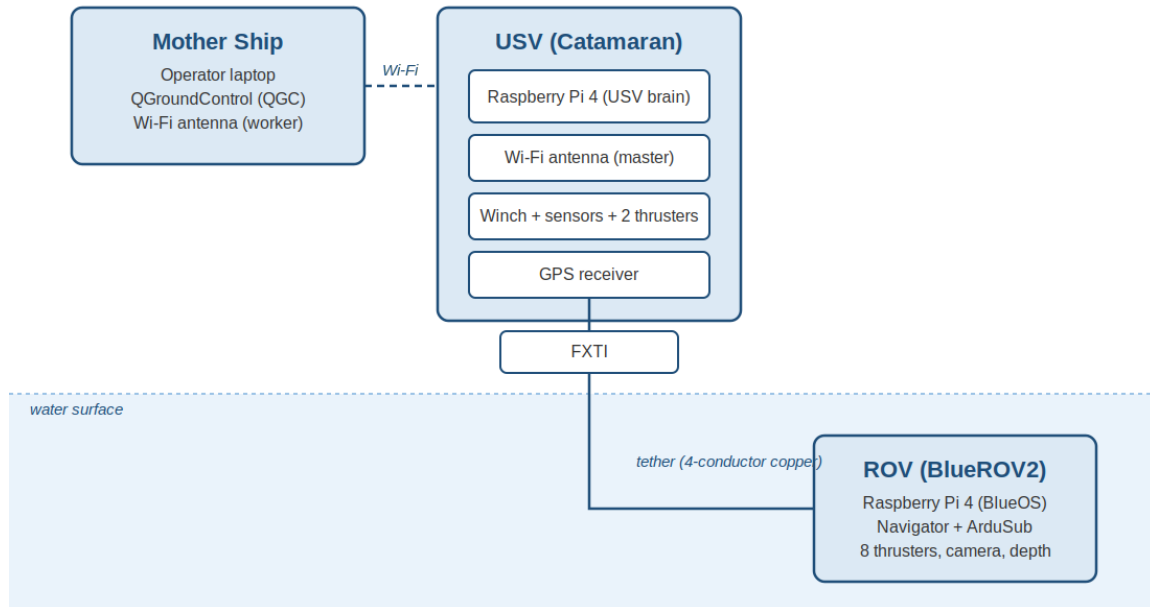


Figure 1: System overview scheme

3.1.1 USV Subsystems

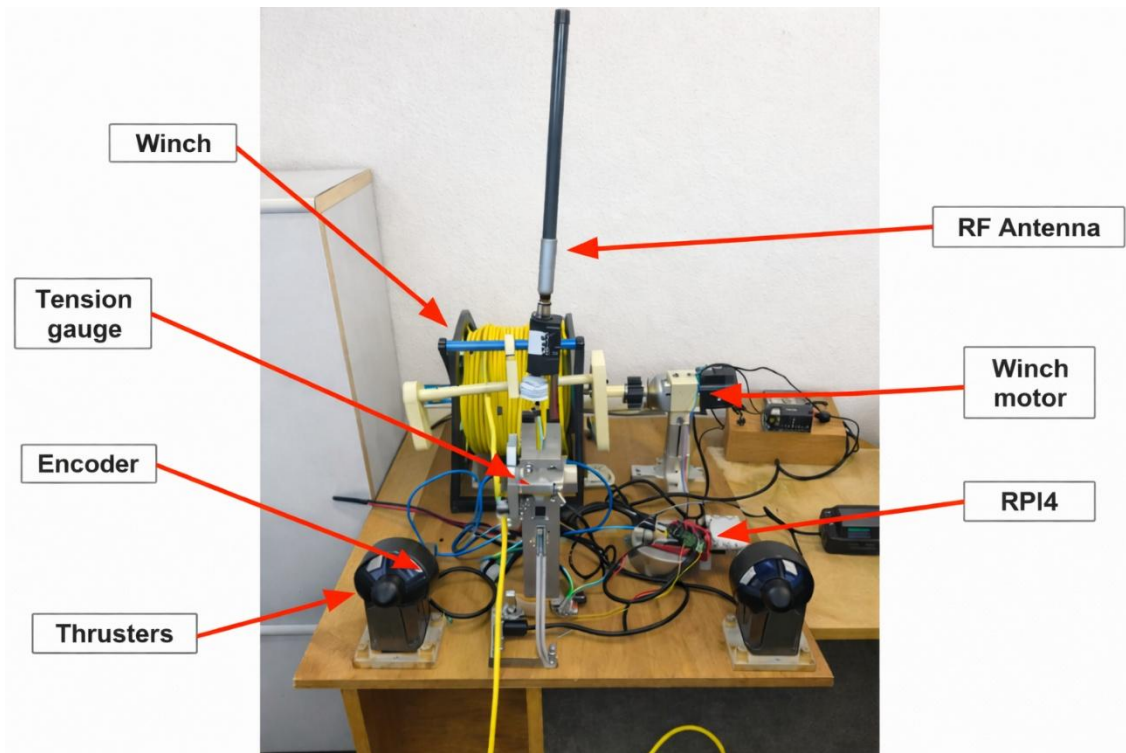


Figure 2: USV Overview Picture

The USV serves as the autonomous tracking relay and network gateway (Figure 1).

- **Central Computing & Middleware:** The primary onboard computer is a Raspberry Pi 4 running Ubuntu 20.04.6 LTS (Long-Term Support - 64-bit). The software architecture operates on a distributed ROS 2 (Foxy) middleware, where each sensor and actuator runs as an independent node in its own thread (Figure 2).
- **Propulsion System:** Driven by two Blue Robotics T200 thrusters mounted on a dual-HDPE (High-Density Polyethylene) catamaran float chassis. The thrusters are controlled by bidirectional Basic ESCs (Electronic Speed Controller) utilizing a 50Hz hardware-timed Pulse-Width Modulation (PWM) signal, generated continuously by a background pigpio daemon to ensure timing stability.
- **Winch & Tether Management:** The active payload is managed by an integrated stepper servo motor (StepperOnline iSV57T-130S, NEMA (National Electrical Manufacturers Association) 23) featuring a built-in 16-bit encoder and driver. It is paired with an EG23-G10-D8 planetary gearbox (10:1 ratio) and configured via microstepping to require 3200 pulses per revolution, providing high-torque, precision control over tether pay-out and reel-in.
- **Kinematic Tracking Sensors:**
 - **Tether Angle:** Monitored by a CUI AMT112S-V capacitive incremental rotary encoder featuring 8192 ticks/revolution with quadrature decoding, powered by a 3.3V logic rail.
 - **Tether Tension:** Measured by a mechanical load cell in a three-pulley configuration, amplified and digitized by an HX711 24-bit ADC module connected via a 2-wire bit-banged protocol.
- **Navigation & Inertial Data:** An onboard u-blox GPS receiver communicates via USB (Universal Serial Bus) using the UBX (u-blox proprietary binary GPS protocol) binary protocol, supplemented by a 6-DOF IMU (Inertial Measurement Unit) for attitude awareness.

3.1.2 ROV (Remotely Operated Vehicle) Subsystems



Figure 3: ROV picture

The underwater agent executes the autonomous missions.

- **Vehicle Platform:** A commercially available Blue Robotics BlueROV2, specifically upgraded to the **Heavy Configuration** (Figure 3).
- **Propulsion:** Eight vectored thrusters providing full 6-Degree-of-Freedom (6-DOF) control (surge, sway, heave, roll, pitch, yaw) for enhanced underwater stability.
- **Computing & Control:** Houses a dedicated Raspberry Pi 4 ("ROV Pi") running BlueOS, interfaced with a Navigator flight controller running ArduSub firmware.
- **Sensing:** Equipped with a primary high-definition camera for real-time video streaming, an internal IMU, and a Bar30 high-resolution depth sensor.

3.1.3 Communication & Networking Infrastructure

- **Surface RF Link (Mother Ship ↔ USV):** A long-range Wi-Fi bridge utilizing a paired Master/Worker antenna setup (e.g., Ubiquiti) operating on 2.4/5 GHz frequencies (Figure 6).
- **Underwater Link (USV ↔ ROV):** A physical 4-conductor copper tether terminating at the USV into a Fathom-X Tether Interface (FXTI – Figure 5). The FXTI (Fathom-X

Tether Interface) acts as a media converter, feeding ethernet data via USB into the USV Raspberry Pi (Figure 4).

- **Data Routing:** MAVLink telemetry (Port 14550) and UDP (User Datagram Protocol) Video streams (Port 5600) are routed through the USV Pi over the Wi-Fi link directly to the QGroundControl software on the operator's laptop.



Figure 4: Fathom cable



Figure 5: FXTI



Figure 6: RF Antenna

3.2 Key Engineering Changes Since PDR

As development transitioned from theoretical design to physical integration and laboratory testing, several critical adjustments were made to the system's architecture to ensure reliability and performance.

3.2.1 Power Supply and PoE (Power over Ethernet) Integration

- **PDR Proposal:** The original design assumed standard, independent mobile power supplies sized for the USV's communication antennas.
- **Final Implementation:** During integration, it was identified that the RF antennas strictly required simultaneous power and data transmission, which the RPi4 (Raspberry Pi 4) cannot natively provide. To solve this during the dry lab testing phase, standard wall-powered PoE Injectors were utilized to merge power and data streams. For the upcoming wet trials and field operations, this architecture is being upgraded to include a PoE-enabled Network Switch onboard the USV. The FXTI will connect directly to this switch, providing a fully integrated, battery-driven power and data link.

3.2.2 Network Routing, Static IPs, and Overcoming Restrictions

- **PDR Proposal:** The preliminary design assumed a straightforward, plug-and-play network bridging between the control laptop, USV, and ROV.
- **Final Implementation:** Severe IP (Internet Protocol) address conflicts arose when integrating multiple networked devices, exacerbated by the transition from a software simulator back to the physical hardware. The antennas and the ROV operated on different subnets, breaking communication. To resolve this, a comprehensive network reorganization was executed using Static IP Allocation. The RF antennas were reconfigured on a unified subnet, and the USV's RPi4 was explicitly configured to act as the official network bridge routing traffic. Furthermore, to bypass university public Wi-Fi security protocols that blocked RPi4 configuration, local networks were established via mobile hotspots and direct LAN (Local Area Network) connections to personal routers.

3.2.3 Implementation of the RTL (Return-to-Launch) Failsafe State

- **PDR Proposal:** The Mission Manager was originally designed to handle basic operational modes such as Manual, Auto, and Stop.

- **Final Implementation:** An improved State Machine was developed, introducing a dedicated RTL failsafe mechanism. If the system detects a critical fault - such as communication loss, tension overload (>135 kg), or battery failure it forces a transition to RTL. In this state, the ROV automatically ascends to a safe depth, the USV holds baseline tether tension, and the USV navigates both vehicles back to the original launch GPS coordinates before shutdown.

3.2.4 Three-Zone Kinematic Control Logic

- **PDR Proposal:** The control loop algorithm was broadly defined to process physical parameters to determine generic "movement vectors" for the USV.
- **Final Implementation:** To prevent over-correction and ensure smooth tracking, the Follower node was upgraded from a generic vector output to a formal Proportional (P) Controller featuring non-linear stability zones. Let the error $e(t)$ be the measured cable angle $\theta(t)$, and $u(t)$ be the required PWM output to the thrusters. The baseline control equation is:

$$(1) \quad u(t) = K_p \cdot e(t) + u_{neutral}$$

The proportional gain multiplier (K_p) was determined empirically during dry-lab bench testing. It was tuned specifically to map the maximum expected angular deviation (90°) to the maximum allowable PWM signal ($1700 \mu s$) without inducing system overshoot.

Furthermore, to prevent high-frequency oscillation (chattering) caused by sensor noise and surface wave action, a non-linear stability technique—a dead-zone—was integrated. The resulting three-zone response logic is defined mathematically by the following piecewise function:

$$(2) \quad u(t) = \begin{cases} 0 & \text{if } |\theta| \leq 10^\circ \\ K_p \cdot (\theta - 10^\circ) & \text{if } 10^\circ < |\theta| \leq 30^\circ \\ \text{Differential Thrust} & \text{if } |\theta| > 30^\circ \end{cases}$$

- **Dead Zone (0° – 10°):** Both thrusters remain at neutral ($u(t) = 0$) to ignore minor deviations.
- **Single-Thruster Zone (10° – 30°):** Only the thruster on the side of the cable pull is driven forward proportionally, while the other remains neutral.
- **Differential Zone (30° +):** Maximum turning authority is applied by driving one thruster forward and the opposite in reverse.

3.2.5 GPS Injection and Relative Positioning Validation

- **PDR Proposal:** The initial design noted that GPS would be used solely for USV station-keeping.
- **Final Implementation:** Because GPS signals do not propagate underwater, the USV was programmed to act as a localized positioning satellite for the ROV. The position estimator node actively computes the absolute ROV coordinates by taking the USV's GPS fix and applying Pythagorean offsets derived from the deployed tether length and encoder angle. This absolute position is published and injected into the MAVLink stream, allowing the mission manager and the operator to track the ROV.

3.2.6 Hardware-Timed PWM Generation (pigpio Daemon)

- **PDR Proposal:** The system assumed that the Raspberry Pi could drive the thruster operations directly through standard software commands.
- **Final Implementation:** During bench testing, it was found that pure-Python control loops suffer from OS-level interruptions, resulting in jittery, unstable PWM signals that caused the thrusters to fluctuate. To guarantee the strict 50Hz timing required by the Blue Robotics ESCs, the pigpio daemon (pigpiod) was integrated into the operating system. This background process generates precise, hardware-timed PWM signals autonomously, independent of the Python control logic.

3.2.7 Mechanical and Algorithmic Overhaul of the Winch System

- **PDR Proposal:** The preliminary design assumed the existing legacy winch mechanism and its baseline control code were functional and ready for integration.
- **Final Implementation:** Physical integration revealed that the original Python winding algorithm operated too slowly (latency of about 2 sec). It was heavily optimized: by removing time-delay bottlenecks and isolating the motor-control loop into its own thread, the achievable motor speed was increased from an unusable rate to approximately 1200 RPM (Revolutions Per Minute) — 120 RPM at the spool through the 10:1 gearbox. In practice, the winch runs well below this ceiling, releasing and retracting cable at ~16 RPM at the spool.

3.2.8 Thruster Polarity Reversal Delay (Preventing Motor Stalling)

- **PDR Proposal:** The system assumed standard control logic with instant thruster response to generic movement vectors.

- **Final Implementation:** During bench and integration testing, rapid switching from forward to reverse thrust caused significant current transients (regenerative voltage) that triggered the motor controllers' safety shutdowns. To mitigate this, the thruster control logic was modified to introduce a programmed software "neutral" delay of several seconds between forward and reverse commands. This acts like a traditional transmission, allowing the current to dissipate safely and preventing the thrusters from crashing.

4. Proposed Solution and Engineering Design Approach

4.1 General Problem-Solving Approach

The core strategy relies on the "Leader-Follower" (Master-Slave) control paradigm, where the ROV acts as the autonomous "Master" executing the underwater mission, and the USV acts as the "Slave" tracking it and serving as an autonomous communication relay.

Instead of utilizing expensive and complex acoustic positioning systems (e.g., Ranger) to identify the ROV's location relative to the USV, the project adopts a **mechanical-kinematic approach**: utilizing the physical tether itself as a feedback mechanism. By measuring the tether's tension and deviation angle, the control system deduces the ROV's movement vector and translates it into propulsion and winch commands, maintaining optimal relative positioning and preventing cable damage.

4.2 Software Architecture and Block Diagram (ROS 2)

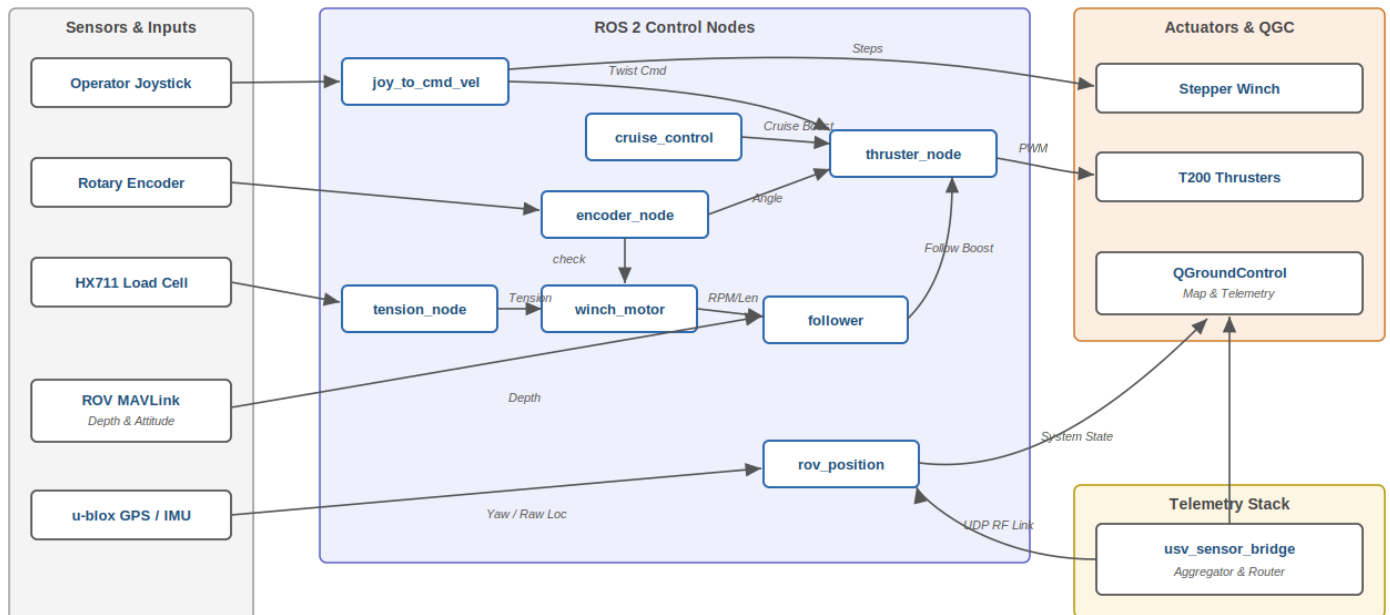


Figure 7: Software Architecture and Block Diagram

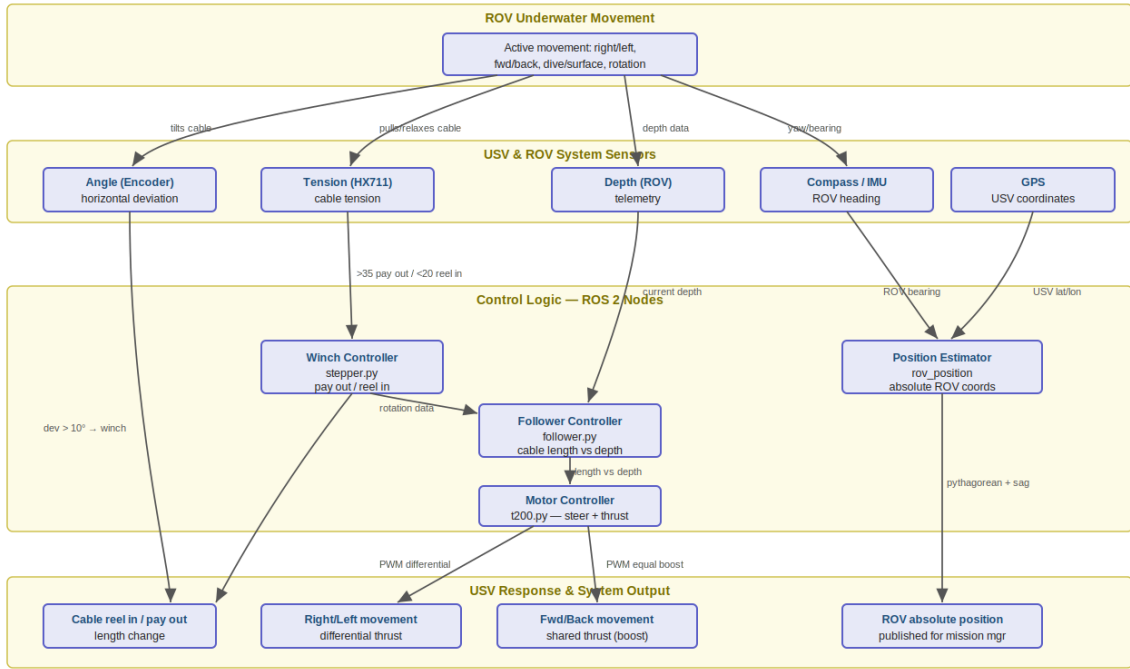


Figure 8: System FW flow chart

The USV's software system was developed within the ROS 2 Foxy middleware, enabling a distributed architecture where each sensor and actuator functions as an independent Node running in a separate thread, communicating via Topics (Figure 8).

Control Nodes Block Diagram Description: The architecture is divided into three main sections: Sensors & External Inputs, ROS 2 Control Nodes, and Actuators & QGC (QGroundControl) (Figure 7).

- **Sensors (Inputs):** Data is acquired from a Rotary Encoder, an HX711 Load Cell, a u-blox GPS/IMU (MPU6050), and ROV MAVLink telemetry.
- **Control Nodes:**
 - `encoder_node` and `tension_node`: Process raw sensor signals (angle and tension) and publish the calculated values.
 - `winch_motor`: Receives tension data to calculate required stepper motor steps and publishes the deployed cable length to the position and follower nodes.
 - `follower` and `cruise_control`: The core tracking logic. The follower calculates dynamic boost based on ROV depth vs. cable length, while `cruise_control` provides a constant forward boost when the cable angle is straight ($<10^\circ$ deviation).

- `rov_position`: The localization algorithm that takes the USV's raw GPS fix, cable length, and angle to compute the absolute ROV coordinates via Pythagorean kinematics.
- `thruster_node`: Aggregates all thrust vectors (Follow, Cruise, and manual Twist commands from `joy_to_cmd_vel`) into PWM signals for the T200 thrusters.
- **Telemetry Stack (Outputs)**: All outputs are routed through the `usv_sensor_bridge`, formatting data into a UDP stream transmitted over the RF link to QGC.

4.3 Flowchart - Tracking Logic and State Machine

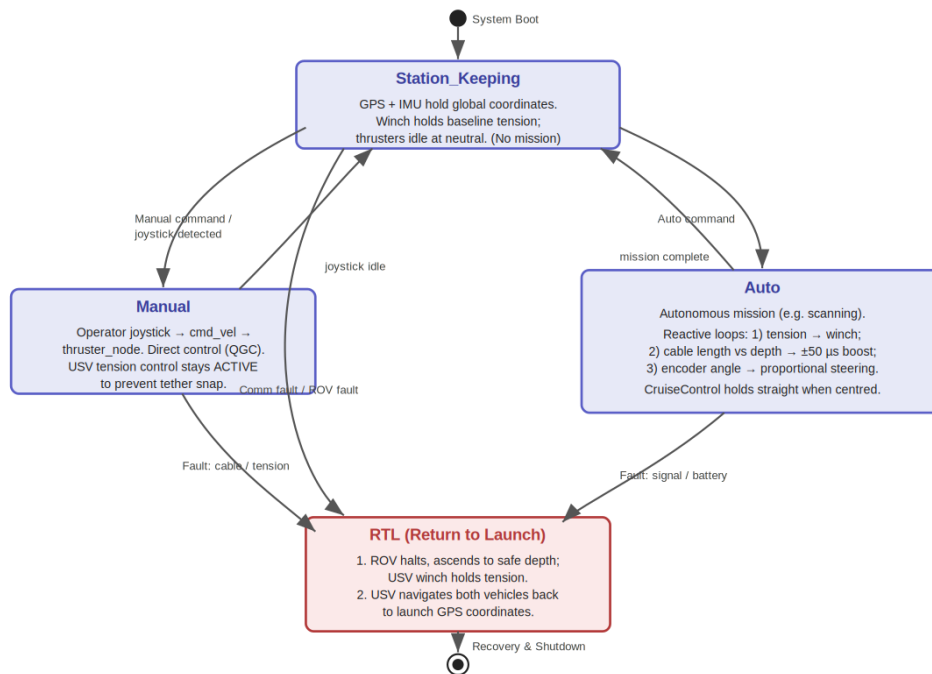


Figure 9: Tracking Logic and State Machine

The system operates as a robust, safety-oriented state machine divided into four primary states:

4.3.1 Station Keeping:

The default boot state. The USV maintains global GPS coordinates, the winch holds baseline tension, and the thrusters remain neutral. Operator joystick input shifts the system to Manual, while a MAVLink command initiates Auto (Figure 9).

4.3.2 Manual:

`cmd_vel` inputs are injected into the thruster node. Winch tension control remains active to prevent tether snapping during sudden acceleration (Figure 9).

4.3.3 Auto:

Divided into two sub-states:

- **Reactive:** Runs three parallel loops. The primary tension loop controlling the winch acts as a non-linear ON/OFF controller with hysteresis. To ensure mechanical stability and prevent motor chattering caused by wave-induced tension spikes, a 15 kg hysteresis dead-band was designed into the control logic. Let T represent the measured tether tension in kilograms. The winch state is determined by:

$$(3) \quad \text{Winch State} = \begin{cases} \text{Pay Out} & \text{if } T \geq 35 \\ \text{Reel In} & \text{if } T \leq 20 \\ \text{Hold (Previous State)} & \text{if } 20 < T < 35 \end{cases}$$

Alongside this, a length-vs-depth loop provides forward/backward boost, and an encoder loop translates horizontal deviation ($> 10^\circ$) into proportional differential steering (Figure 9).

- **CruiseControl:** Engages when angle deviation is $< 15^\circ$ and tension is nominal (20-35 kg), providing a constant forward boost ($\sim 30\mu\text{s}$) to advance steadily above the ROV without zig-zagging.

4.3.4 RTL:

An automatic failsafe triggered by critical faults (RF timeout, tension > 135 kg, power loss, or ROV internal leakage). The ROV halts and ascends to a safe depth, the winch holds tension, and the USV navigates both platforms back to the launch GPS coordinates (Figure 9).

4.4 Electronic Circuit Design, Communication, and Hardware Timing

4.4.1 Central Computing & PWM Generation:

Controlling the ESCs requires strict 50Hz hardware-timed PWM. Because pure-Python loops suffer from OS-level interruptions that cause motor stuttering, the pigpio daemon (pigpiod) was integrated into the OS (Operating System) to generate precise hardware-timed PWM signals independently from the ROS 2 logic.

4.4.2 Logic Conversion & Sensors:

The AMT112S-V encoder natively outputs 5V logic. To protect the RPi4's GPIO (General-Purpose Input/Output) pins, it is powered from the 3.3V rail, ensuring safe

3.3V output signals. The tension load cell utilizes a 24-bit HX711 ADC over a 2-wire protocol, while the main winch driver is powered by a dedicated 24V supply.

4.4.3 Network & PoE Delivery:

The long-range RF Ubiquiti antennas require simultaneous data and power. The electronic solution integrates a PoE-enabled Network Switch onboard the USV. The FXTI media converter routes data through the RPi4, which acts as a gateway, into the PoE switch that powers the RF antennas.

4.5 Design Considerations

Several critical considerations dictated the final engineering solutions:

4.5.1 Master/Slave Architecture:

The ROV serves as the primary reference (Master), while the USV acts as the autonomous follower (Slave). This hierarchy ensures the USV dynamically positions itself above the ROV, preventing tether entanglement and maintaining communication.

4.5.2 ROV GPS Injection:

Rather than injecting the raw USV surface coordinates directly into the ROV, the system calculates the ROV's true underwater position. To achieve this, the position estimator node processes several known variables: the USV surface position (x_{usv}, y_{usv}) derived from the onboard GPS, the ROV depth (z_{rov}) from MAVLink telemetry, the deployed tether length (L) tracked by the winch encoder, and the horizontal cable heading angle (θ) measured by the rotary encoder.

First, the horizontal planar distance (d) between the USV and ROV is calculated using the Pythagorean theorem:

$$(4) \quad d = \sqrt{L^2 - z_{rov}^2}$$

Next, the absolute geographic coordinates of the ROV are computed using standard trigonometric projection:

$$(5) \quad x_{rov} = x_{usv} + d \cdot \cos(\theta)$$

$$(6) \quad y_{rov} = y_{usv} + d \cdot \sin(\theta)$$

To account for the physical bowing of the underwater cable due to hydrodynamic drag and gravity, a scaling sag factor—representing a basic catenary curve approximation—

is applied to the tether length (L) before the final computation. This precise, mathematically derived positional data is then continuously injected into the ROV's MAVLink stream to satisfy the Extended Kalman Filter (EKF) requirements for autonomous navigation.

4.5.3 Control Logic & Thresholds:

To prevent system oscillation and mechanical wear, strict sensor-actuator dead-bands were established. For example, USV thrusters only activate when the tether angle exceeds a 10° threshold, and the autonomous winch system spools or unspools based on strictly predefined tension-force limits.

4.5.4 RTL:

We designed this autonomous failsafe to trigger during critical faults (e.g., RF timeout, leaks) to guarantee the safe, simultaneous recovery of both platforms. Upon activation, the ROV ascends to a predefined shallower towing depth to significantly reduce hydrodynamic drag and mechanical tether stress. The USV then navigates back to the initial launch GPS coordinates at a restricted maximum speed to prevent overloading the towing hardware. Simultaneously, our active winch slack management eliminates the risk of the cable dragging on the seabed or entangling in the USV's propellers.

5. Final Product Acceptance Testing

This section outlines the testing array, measurement methodologies, and the final measured performance of the integrated USV-ROV system. The acceptance testing was conducted using a custom automated validation framework designed to sequentially verify physical hardware interfaces, software control loops, and full-system communication integration.

5.1 Test Array and Measurement Methodology

The test array was designed to evaluate the system across three distinct operational layers:

5.1.1 Hardware-in-the-Loop (HIL) Verification:

Testing the direct physical connection between the central computer and the peripheral sensors/actuators. This ensures that raw data is read correctly (e.g., accurate tare/calibration of weight and angle) and that electrical commands reach the motor controllers without OS-level interruptions.

5.1.2 Autonomous Control Logic Validation:

Testing the mathematical decision-making of the system. Simulated physical conditions (such as a sudden manual pull on the cable or angular deflection) are injected into the system on the test bench to verify that the tracking algorithms generate the correct, safe propulsion commands.

5.1.3 End-to-End System Integration:

Testing the communication bridge between the surface vehicle (USV) and the underwater vehicle (ROV) to verify the successful execution of telemetry, flight mode changes, and hardware maneuver commands over the MAVLink network.

5.2 Measured Product Performance

The system successfully passed the comprehensive acceptance testing in the lab, demonstrating reliable hardware interaction and stable autonomous logic. The following details the objectives and results across the primary subsystems:

5.2.1 USV Hardware Bring-up and Sensor Verification

- **GPS Connectivity:** Tested to ensure the receiver establishes a stable connection and streams continuous, valid location data, which is critical for the ROV's localization. Result: The module successfully initialized and maintained a high-quality, uninterrupted data stream to the system.

- **Tension and Angle Sensors:** Tested to verify that the load cell and rotary encoder calibrate properly upon startup and provide accurate, drift-free readings. Result: Both sensors successfully established a zero-baseline. The tension meter accurately measured dynamic weight changes, and the encoder precisely tracked angular deviations.
- **Actuator Control (Thrusters and Winch):** Tested to confirm that the motor controllers accurately process directional and speed commands. Result: The winch motor seamlessly toggled between pay-out and reel-in directions, and the USV thrusters accurately executed neutral, forward, and reverse propulsion signals based on the PWM inputs.

5.2.2 Autonomous Tracking Logic Validation (Bench Testing)

The autonomous tracking algorithms were validated on the test bench under manual physical stimuli designed to simulate the forces expected from the ROV. This ensured the USV logic computes and generates the correct propulsion commands in response to tether variations:

- **Dynamic Tension Response:** Tested the interaction between the tension sensor and the winch motor by manually pulling the tether. Result: The system accurately read the tension spikes and successfully triggered the failsafe logic, automatically commanding the winch motor to pay out cable the moment the load exceeded the normal operating threshold (>35 kg) to maintain safe tension and prevent snapping.
- **Proportional Steering Logic:** Tested the software link between the cable angle encoder and the USV thruster commands. Result: The system successfully validated the steering formula; when the cable angle was manually deflected beyond the dead-zone, the controller successfully dispatched the exact PWM commands to the ESCs required to execute differential corrective thrust.
- **Cruise Control Safety Gates:** Tested the logical conditions under which the USV commands a constant forward cruising speed. Result: The system correctly authorized the forward boost command only when the cable was straight and tension was nominal. The system demonstrated the ability to instantly and completely cut motor power commands if the angle was deflected or tension was manually spiked during the test.

5.2.3 Full System Integration and ROV Control

The final phase of acceptance testing validated the USV's ability to act as the primary operational gateway and control the physical ROV hardware.

- **Telemetry and Attitude Streaming:** Tested the communication data bridge between the operator and the ROV. *Result:* The system established a highly stable UDP connection, continuously receiving real-time orientation (yaw/pitch/roll) and status updates from the ROV at a steady 4Hz rate.
- **Flight Mode Management:** Tested the ability of the USV to change the ROV's operational states remotely. *Result:* The system successfully commanded the ROV to arm, disarm, and transition smoothly between different flight modes (e.g., transitioning from Manual to Stabilize).
- **Physical Execution (Full Hardware Demo):** The final acceptance test verified full physical control over the ROV's propulsion and payloads. *Result:* The system successfully executed a comprehensive thrust sequence, verifying ROV motor activation in all eight spatial directions. Furthermore, it validated control over auxiliary payloads, smoothly operating the ROV's external lighting system and fully opening and closing the robotic gripper upon remote command.

5.3 Conclusion of Acceptance Testing:

The measured performance on the test bench confirms that the mechanical architecture, the control logic (translating simulated physical tether variations into accurate motor commands), and the MAVLink communication bridge meet the project's stringent operational requirements. The system has successfully demonstrated complete hardware control, correct logic computation, and robust failsafe execution in a dry laboratory environment.

5.4 List of Tests:

Test Name & Description	Executed (V / X)	Result & Explanation
Tension Sensor (HX711): Verify zero-baseline calibration and dynamic weight measurements	V	Pass
Angle Encoder: Verify angle initialization (0°) and rotational tick translation	V	Pass
GPS Receiver: Verify continuous location data stream to the USV over USB	V	Pass
Thruster PWM Output: Verify pigpiod generates strict hardware-timed PWM signals for motor ESCs	V	Pass
Winch Stepper Frequencies: Verify step pulse generation and dual-speed winding without OS interruption	V	Pass
GPS Injection: Verify normal GPS data injection utilizing the Pythagorean calculation	V	Pass
Proportional Steering: Verify differential thrust engages when the cable angle exceeds the 10° dead-zone	V	Pass
Cruise Control Gates: Verify forward boost (+30μs) only activates under nominal tension and straight angle (<15°)	V	Pass
Thruster Reversal Delay: Verify system handles rapid forward-to-reverse commands	V	Pass
Dynamic Tension Response: Verify the winch automatically pays out cable when tension >35 kg	X	Not Executed: Not tested due to a broken mechanical pin between the winch motor and the spool
ROV Telemetry Stream: Verify stable UDP MAVLink connection for attitude (yaw/pitch/roll) data	V	Pass
Flight Mode Management: Verify USV can remotely change ROV states (Arm, Disarm, Manual, Stabilize)	V	Pass: Verified via opcode from VSCode (USV terminal) and the physical operator controller

RF Link Range (100 m): Verify stable data/video throughput over the required 100-meter range	X	Not Executed: Tested successfully at 3 meters in the lab, but the full 100m range was deferred due to physical lab space limitations and AC power dependencies
Full ROV Hardware Demo: Verify 8-direction thrust, external lights dimmer, and robotic gripper via USV relay	V	Pass
Winch Lead-Screw Overhaul: Verify smooth cable winding after replacing the broken legacy mechanism	V	Pass
RTL Failsafe: Verify RTL state halts ROV and commands USV return on critical faults	X	Not Executed: Not tested during this phase
Pool Test / Buoyancy: Verify physical buoyancy, tension meter water drag, and basic movement in water	X	Not Executed: Deferred due to wet-test coordination difficulties and delayed by broken mechanical components
Autonomous Polygon Scan: Verify ROV successfully flies a 20x20m pre-planned Auto-mode mission underwater	X	Not Executed: GPS-injection bridge logic passed in software on the bench, but physical execution requires open-water sea trials
Wind, Waves & Currents: Verify system tracking, catamaran station-keeping, and physical stability against surface wind loads, waves, and strong underwater currents	X	Not Executed: Deferred to future open-water sea trials
GPS Outages & RF Interference: Verify system behavior, data recovery, and failsafe triggering during GPS signal loss and active RF interference	X	Not Executed: Deferred to future field testing
Open-Water Tether Dynamics: Verify control loop stability when subjected to real-world underwater tether drag, catenary cable behavior, and variable slack in open water conditions	X	Not Executed: Deferred to future wet trials
Localization Uncertainty & Scalability: Verify system accuracy under relative localization uncertainty and assess network/hardware scalability	X	Not Executed: Requires extensive field data gathering and will be tested in future iterations

Table 2: List of tests

6. Problems and Solutions

During the design, development, and operational phases of the project, several technical and logistical challenges arose, requiring significant engineering adjustments. The following outlines the primary problems encountered and the solutions implemented to overcome them:

6.1 Connecting and Configuring Wi-Fi on the Raspberry Pi 4

- **Problem:** We experienced significant difficulties establishing a stable Wi-Fi connection and configuring the network on the RPi4, particularly due to security protocols that blocked access to the university's public Wi-Fi network.
- **Solution:** We performed a clean OS installation (re-flash) on a new RPi4 and install all previous years' code to ensure a fresh start. To bypass the university network restrictions, we established local networks using a mobile hotspot or by physically connecting the RPi4 to a personal home router via LAN.

6.2 Remote Work and Hardware-Simulator Transition

- **Problem:** The project faced a dual challenge. Initially, the security situation severely restricted regular laboratory access, halting physical hardware integration. Subsequently, the actual transition of our developed systems from the software simulator back to the physical ROV proved highly problematic, as the differing network environments caused severe IP mismatch conflicts and communication breakdowns.
- **Solution:** To bypass the physical lab restrictions, we deployed a local ROV simulator, focusing our efforts on developing the RF antenna protocols and optimizing the GPS receiver setup where physical movement was not strictly required. To overcome the transition hurdles upon returning to the lab, we completely overhauled the network architecture, reconfiguring the static IP address allocations across the entire system to establish a proper, unified communication bridge between the control station and the physical hardware.

6.3 IP Address Conflicts and Network Routing

Multiple networked devices (RF antennas, ROV, control laptop) initially operated on mismatched subnets, blocking simultaneous communication. This was resolved with a full Static IP reallocation - [see 3.2.2 for details](#).

6.4 RF Interference Between Communication Antennas

- **Problem:** During initial dry testing, the RF data link between the antennas failed completely. Troubleshooting revealed that placing the antennas too close together caused severe mutual RF interference and signal jamming.
- **Solution:** We relocated the antennas to maintain a minimum physical separation of 3 meters. This distance successfully mitigated the electromagnetic interference, allowing for a stable data link between the operator's laptop and the USV's onboard RPi4.

6.5 Power over Ethernet (PoE) Supply for Antennas

The RF antennas required simultaneous power and data delivery, which the RPi4 cannot natively provide. This was solved in two phases - a PoE Injector for lab testing, a PoE Switch for field trials - [see 3.2.1 for details](#).

6.6 USV Motor Stalling During Sudden Direction Changes

Rapid forward/reverse thrust switching caused current spikes that triggered motor-controller safety shutdowns. A programmed neutral delay between direction changes resolved this - [see 3.2.8 for details](#).

6.7 ROV Restriction on Entering Autonomous 'Auto' Mode

- **Problem:** Our original concept relied on deducing the ROV's underwater position mathematically (USV GPS + Pythagorean offset) and reflecting it in QGroundControl. Although the ROV appeared on the map, the physical ArduSub flight controller failed to arm and enter AUTO mode for waypoint navigation. Attempts to force the location awareness by disabling GPS and compass checks worked on the software simulator but caused critical communication failure ("Connection refused" and "Lost manual control") on the physical hardware. The ROV's Extended Kalman Filter (EKF) actively rejected the data, expecting a physical hardware sensor (like a DVL (Doppler Velocity Log)).
- **Solution:** We developed a dedicated asynchronous software bridge running as a background service (systemd) that successfully translates and injects external GPS data directly into the ROV's EKF without crashing the primary control link. This allowed the ROV to achieve a stable 3D Fix and successfully execute autonomous missions. The architecture is broken down into four critical stages:

1. Serial Data Parsing: The script opens a serial communication channel to the USB GPS (/dev/ttyACM0). It filters the continuous NMEA data stream specifically for \$GNGGA (NMEA sentence identifier for GNSS Fix Data) sentences to extract precise altitude, latitude, and longitude coordinates, parsing the complex text strings into actionable data using the pynmea2 library.
2. MAVLink Formatting: Because the ArduSub flight controller cannot process raw NMEA text, the script translates the decimal coordinates into MAVLink's fixed-point (1e7 degrees) integer format for lat/lon.
3. Targeted EKF Injection: Instead of spoofing a physical GPS sensor (Message #24), the data is packaged into the gps_input_send MAVLink message (Message #232). This explicitly instructs the ROV to accept external location data directly into its navigation filter.
4. Asynchronous UDP Routing: To solve the critical control-link collisions, the system physically isolates the communication channels. The bridge uses a "fire-and-forget" UDP protocol to inject the location data onto a secondary port (Port 14551). This leaves the primary QGC control channel (Port 14550) completely unobstructed.

7. Conclusions and Recommendations

This chapter summarizes the development efforts, analyzes the degree of success in achieving the original project objectives, discusses the pros and cons of the chosen architecture, and provides critical operational recommendations for future development and upcoming field trials.

7.1 Project Success and Objective Achievement Analysis

The evaluation of the project's success is based on the original objectives and Success Metrics defined at its inception, compared against the results of the laboratory acceptance tests:

1. Operator–ROV Decoupling:

- **Achieved?** Yes (Proven in laboratory).
- **Details:** The architecture was successfully implemented, establishing stable bidirectional communication for commands and telemetry without a direct physical tether between the operator and the ROV. However, the original success metric required demonstrating a minimum range of 100 meters. Because the project concluded at the Dry Lab Test phase, the wireless range in an open water environment has not yet been physically proven, though the communication infrastructure (Ubiquiti antennas and UDP routing) is fully capable of supporting it.

2. Autonomous Mission Execution by the ROV:

- **Achieved?** Yes (Software/Infrastructure level), No (Field execution level).
- **Details:** The flight controller was updated, and the ROV configuration now supports autonomous missions following the development of the software "bridge" that injects GPS data into the EKF filter. However, because the project was halted at the lab validation stage, the physical 20x20-meter polygonal scanning mission was not executed underwater.

3. Autonomous USV Tracking of the ROV:

- **Achieved?** Yes (Under Dry Test laboratory conditions).
- **Details:** A comprehensive tracking logic based on the tension sensor and angle encoder was developed. Bench tests proved that the USV system responds correctly, maintains tether tension limits, and applies corrective directional

thrust. The physical ability of the USV to overcome hydrodynamic drag in the water has not yet been tested in a wet trial.

4. Quantitative Real-Time Video Transmission via USV Relay:

- **Achieved?** Yes.
- **Details:** The USV successfully functions as a communication relay. Video streams and control interfaces reach the QGC software in real-time via the RF channel. Precise bandwidth and latency measurements compared to a direct tether will be completed during wet trials, but laboratory performance is smooth and allows for full control. In the dry lab test, when the antennas were 3 meters away from each other, there was no delay in data, control, or video. Because of the laboratory limitations (power sockets), we weren't able to test the antennas beyond a few meters.

Testing Plan Status Summary:

- **Dry Lab Test:** Passed with full success. Commands from QGC successfully reached the ROV via the USV, and the data stream returned seamlessly.
- **Pool Test / Sea Trials:** Not achieved prior to report submission. The system is logically sound but awaits wet trials for final in-water validation. These trials were not executed due to wet-test coordination difficulties stemming from the security situation over the past year, as well as a lack of a physical component that broke in the subsequent year, which delayed further physical testing.

Project Objectives Table:

Project Objective	Achieved?	Validated Successes (Lab / Software)	Pending Validation (Wet Trials) & Details
Operator–ROV Decoupling	Yes (Proven in laboratory)	Architecture implemented; stable bidirectional communication (commands and telemetry) established without a direct physical tether	Pending: 100m minimum range in an open water environment. Details: Project concluded at Dry Lab phase, though infrastructure is capable
Autonomous Mission Execution (ROV)	Yes (Software level) / No (Field level)	Flight controller updated; GPS-to-EKF software injection "bridge" developed and supports autonomous missions	Pending: Physical 20x20m polygonal scanning mission underwater. Details: Halted at the lab validation stage

Autonomous USV Tracking of the ROV	Yes (Under Dry Test conditions)	Tracking logic developed; bench tests proved the system maintains tension limits and applies corrective directional thrust	Pending: USV physical ability to overcome hydrodynamic drag. Details: Awaits wet trials for in-water validation
Real-Time Video Transmission via USV Relay	Yes	USV successfully acts as a relay; real-time video/control reached QGC with almost zero delay at a 3-meter distance	Pending: Precise bandwidth and latency measurements at range. Details: Testing beyond a few meters was restricted by lab limitations (power sockets)
Pool Test & Open-Water Sea Trials	No (Not Executed)	N/A – System logic and software passed dry tests, but physical water testing was deferred	Pending: Physical buoyancy, hydrodynamic drag, weather resistance (wind/waves), and open-water tether dynamics. Details: Delayed due to wet-test coordination difficulties (security situation) and subsequent broken components

Table 3: Project Objectives Table

7.2 Planned vs. Actual Comparison (Schedule & Budget)

- Schedule (Gantt):** The original plan assumed continuous laboratory work culminating in open-sea trials. In practice, the security situation severely restricted lab access during critical integration phases, forcing a temporary shift to remote development and simulation. This caused compatibility gaps and network conflicts upon returning to the physical hardware. Consequently, the project concludes at the Dry Lab proof-of-concept stage, with in-water trials deferred to future development.
- Budget & Costs:** Implementing the RPi4-based routing solution eliminated the need to purchase an expensive hardware network switch. Furthermore, the kinematic approach (using the tether as a sensor) saved tens of thousands of shekels by avoiding expensive underwater acoustic localization sensors (DVL/Ranger) at this stage. However, an unplanned requirement emerged for dedicated DC-powered Power over Ethernet (PoE) equipment, which must be procured for the next project phase.

7.3 Discussion: Pros and Cons of the Proposed Architecture

7.3.1 System Advantages:

1. **Use of Commercial Off-The-Shelf (COTS) Components:** The system is built using simple, widely available COTS products (e.g., RPi4, standard load cells, and classic motor controllers), which are easily programmed and integrated within the ROS 2 environment. This approach significantly eases future upgrades, maintenance, and system improvements without relying on "closed" proprietary hardware.
2. **Algorithmic Redundancy and Safety:** The control state machine incorporates deterministic failsafe mechanisms, including a RTL state triggered by communication timeouts or extreme tether tension. This ensures a guaranteed, synchronized recovery sequence for both platforms, mitigating the risk of vehicle loss or tether failure.
3. **Network Flexibility and Scalability:** Because the USV is configured as an IP-based communication router, it is straightforward to integrate additional components and payloads in the future—such as surface cameras or weather sensors—simply by connecting them to the existing network, without needing to alter the ROV's communication protocols.

7.3.2 System Disadvantages and Limitations:

1. **Mechanical Tether Dependency:** The architecture is physically exposed to tether entanglement in the USV's propellers or marine obstacles. In the event of a physical disconnection, the USV completely loses its ability to track the ROV.
2. **Critical Localization Dependency on the USV:** Relying strictly on deriving the ROV's position from the USV's coordinates makes the platforms completely interdependent. This method is highly sensitive to sea conditions (strong currents, high waves) that can distort the kinematic model (angle and tension). Currently, the architecture does not define sufficient filtering logic to resolve noise in real-world sea conditions.
3. **Hydrodynamic Inefficiency and Power Consumption:** The USV is required to overcome both its own hydrodynamic drag and the pull of the heavy ROV tether. This dual load can lead to significantly higher power consumption than standard surface vessels, consequently shortening the joint mission time.

7.4 Recommendations for Future Work

The current prototype has been validated under dry-laboratory and bench conditions. To achieve full open water operational status (Phase B), the system requires targeted hardware modifications, sensor augmentation, and algorithmic expansions. The following engineering objectives are prioritized based on system dependency and implementation complexity.

7.4.1 Phase 1: Environmental Hardening and Power Autonomy:

These modifications address immediate hardware and physical constraints required prior to initiating open-water testing.

- **Mechanical Reinforcement:** Component failures identified during bench testing (specifically the winch mounting brackets) necessitate structural redesign. Replacement parts must be fabricated utilizing materials with higher tensile strength to withstand dynamic hydrodynamic loads.
- **Enclosure Rating and Weight Distribution:** The internal component layout requires spatial optimization to consolidate sensitive electronics - including the planned future power supply - into a dedicated, splash-resistant enclosure designed to protect against open-water spray and water intrusion. The distribution of these components must be analytically modeled to maintain a balanced center of gravity (CG) across the catamaran hulls to prevent listing.
- **Onboard Power Architecture:** The prototype's current AC power dependency must be eliminated. A localized DC battery array must be characterized and integrated to support the maximum continuous current draw of all subsystems (RPi4, ESCs, stepper motors). This includes the addition of a DC-powered PoE switch to ensure stable voltage regulation for the RF antennas.
- **Hydrodynamic Calibration (Wet Trials):** System parameters validated on the bench must be calibrated against actual fluid dynamics. Initial pool testing is required to isolate and define the tether tension-sensor baseline against water drag. Subsequent open-water trials are required to evaluate the control loop response to stochastic surface wave and wind disturbances.

7.4.2 Phase 2: Sensor Augmentation and Mechanical Reliability

These upgrades focus on improving the physical longevity of the system and establishing true independent localization.

- **Acoustic Localization (DVL Integration):** To eliminate the ROV's navigational dependency on the USV's top-down kinematic estimates, a Doppler Velocity Log (DVL) must be integrated into the ROV. Fusing the DVL velocity vectors with the onboard IMU via an Extended Kalman Filter (EKF) will establish independent Dead Reckoning capabilities.
- **Level-Wind and Tension Sensor Mechanical Interface:** The current tether management system employs a custom-designed level-wind (diamond-screw) mechanism alongside a COTS tension sensor. However, under open-water dynamic loads, the lateral translation of the cable driven by the level-wind pulls the tether out of alignment with the tension sensor's bearings. A mechanical redesign of the interface between the level-wind guide and the tension sensor is required. This redesign must smoothly accommodate the tether's lateral movement while maintaining continuous, properly aligned contact with the sensor bearings to ensure reliable operation in an open-water environment.

8 **Bibliography**

- [1] Braginsky, B., Baruch, A., & Guterman, H. (2016). Tracking of Autonomous Underwater Vehicles using an Autonomous Surface Vehicle with Ranger Interrogator System. OCEANS 2016 MTS/IEEE Monterey, Monterey, CA, USA, pp. 1-5.
- [2] Ailon, A., Zohar, I., & Guterman, H. (2016). Trajectory Tracking and Formation Controls for a UAV Model that Incorporates Dynamic Effects. 2016 The 2nd International Conference on Control, Automation and Robotics (ICCAR), Hong Kong, pp. 1-6.
- [3] Ferreira, B., Coelho, A., Lopes, M., Matos, A., Gonçalves, C., Kandasamy, S., Campos, R., & Barbosa, J. (2017). Flexible Unmanned Surface Vehicles enabling Future Internet Experimentally-driven Research. IEEE, Porto, Portugal, pp. 1-6.
- [4] Wang, J. B., Zeng, C., Ding, C., Zhang, H., Lin, M., & Wang, J. (2022). Unmanned Surface Vessel Assisted Maritime Wireless Communication Toward 6G: Opportunities and Challenges. IEEE Wireless Communications, pp. 1-8.
- [5] Cocco, E., Delea, C., Steinmetz, F., Francescon, R., Signori, A., Au, C. N., Campagnaro, F., Schneider, V., Favaro, F., Oeffner, J., Renner, C., & Zorzi, M. (2022). System Architecture and Communication Infrastructure for the RoboVaaS project. IEEE Journal of Oceanic Engineering, pp. 1-53.

9 Appendix

9.1 Task Management

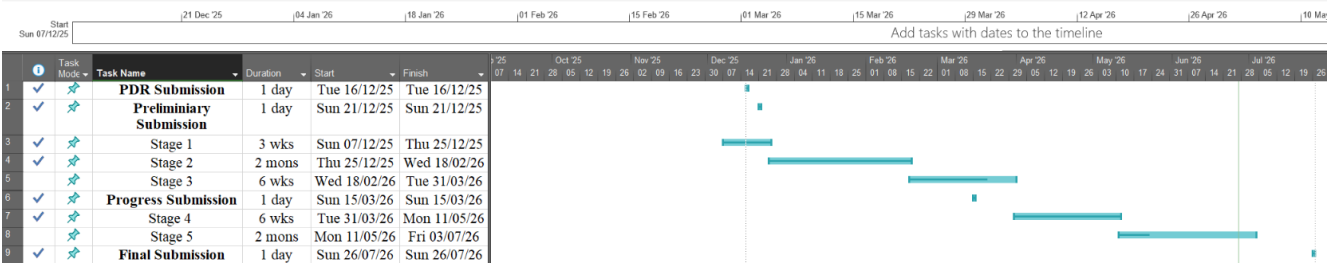


Figure 10: Task management

9.2 User Manual

SYSTEM USER MANUAL

Hovering Autonomous Underwater Vehicle (HAUV) System

ROV • USV • Surface Relay • Tether • Ground Control

Project p-2026-031

Ben-Gurion University of the Negev

Faculty of Engineering Sciences — Dept. of Electrical and Computer Engineering

Students: Guy Hatzir · Maayan Sarig

Supervisor: Prof. Hugo Guterman

1. System Overview

This manual describes the complete system used in the HAUV (Hovering Autonomous Underwater Vehicle) project: a Remotely Operated Vehicle (ROV) connected by a tether to an Unmanned Surface Vehicle (USV), which in turn relays communication to a remote operator (“Mother Ship”) over a long-range RF network. The USV autonomously follows the ROV by sensing the tether’s tension and angle and adjusts its position using two thrusters and a winch.

Further documentation and information in the [Technical Manual](#) below.

1.1 High-Level Architecture

The system is composed of three physically separate platforms, linked by two communication channels:

Platform	Role	Key Hardware
Mother Ship	Mission planning, operator console	Laptop running QGroundControl, RF antenna
USV	Communication relay & autonomous follower	Raspberry Pi 4, two T200 thrusters, winch, sensors, Wi-Fi & GPS antennas
ROV (BlueROV2)	Autonomous underwater vehicle	Raspberry Pi 4, Navigator flight controller, eight thrusters (Heavy configuration), camera, depth sensor

Table A.1: System platforms and their roles

The two communication links are:

- Surface link (Ship ↔ USV): long-range Wi-Fi, 2.4/5 GHz, used for MAVLink telemetry, control and video.
- Underwater link (USV ↔ ROV): a tethered copper cable link (4-conductor tether wires) carrying MAVLink commands and video to and from the ROV.

Note: Throughout this manual, “the USV Pi” or “USV brain” refers exclusively to the Raspberry Pi 4 mounted on the USV catamaran. It is a separate physical unit from the ROV’s own Raspberry Pi 4 (“the ROV Pi”), which runs BlueOS/ArduSub and should not be modified as part of USV work.

Note: Typical battery runtime (ROV + USV combined, with an 18 Ah pack): approximately 2–3 hours under normal use, or 4–6 hours under light use. Plan mission duration accordingly.

1.2 Network Addresses

Each device on the system has a fixed IP address on the network:

Device	IP Address
ROV	192.168.2.2
USV	192.168.2.3
Laptop (operator computer) * Each operator's laptop needs its own IP address config	192.168.2.150
Antenna - master (USV side)	10.0.0.10
Antenna - worker/slave (operator side)	10.0.0.11

Table A.2: Network IP addresses

[More about the antennas](#)

2. Component List, System Assembly, and Power-Up

This section lists every physical component in the system, grouped by which side of the tether it belongs to, followed by a step-by-step procedure for assembling the system and powering it up safely from a cold start.

2.1 List of Components

ROV Side

- ROV - Blue Robotics BlueROV2
- Tether spool
- Short tether cable
- USB Type-B to USB Type-A cable

USV Side

- USV platform - wood board
- Raspberry Pi 4 (USV Pi)
- Mounted tether spool + driver (winch)
- Thrusters - T200 ($\times 2$)
- Tension gauge - HX711 + load cell
- Angle encoder
- GPS antenna and receiver
- Antenna + PoE ($\times 2$)
- FXTI (Fathom-X Tether Interface)

External

- Thruster power source (bench power supply)
- Winch driver power source (bench power supply)
- Operator computer

2.2 System Assembly - Step by Step

1. Connect the ROV upper tube to the tether spool.
2. Connect the tether spool, via the short tether cable, to the FXTI.

3. Connect the FXTI to the USV Raspberry Pi 4 using the USB Type-B to USB Type-A cable (into the RPi4's blue USB port).
4. Connect the GPS antenna and receiver to one of the USV Raspberry Pi 4's USB ports (any free USB-A port).
5. Connect the thrusters, winch/spool driver, tension gauge, and angle encoder wires to the RPi4 pins, using the GPIO pin map (Table A.3 + Figure A.1).
6. Connect the USV RPi4 to mains power (220 V).
7. Connect the first PoE unit to the USV RPi4 (LAN port) and to its antenna (PoE port), and connect the PoE unit to mains power (220 V).
8. Connect the second PoE unit to the operator computer (LAN port) and to its antenna (PoE port), and connect the PoE unit to mains power (220 V).
9. Connect the USV thrusters to their power supply (12V, 5A; OVP (Over-Voltage Protection) upper limit 14V; UVL (Under-Voltage Lockout) lower limit 0V).
10. Connect the spool/winch driver to its power supply (24V).

Click here for [More detailed information](#)

2.3 Power-Up the System

This sequence starts its own numbering and is performed after the system is fully assembled.

1. Connect the ROV battery to power the ROV - bottom tube to upper tube (this powers up the ROV). You will hear the internal ROV Raspberry Pi 4 turn on.
2. [Pay attention to the ROV battery](#)
3. Click "Output" on both bench power supplies to let power flow to the winch and the thrusters.
4. Power up the two PoE units. On each antenna's radio you will see its signal indicator fill the full color bar (yellow to red); the PoE injector itself only shows a single white power light.
5. If everything is properly connected, wait about 2 minutes. You will hear a different sound from the ROV, indicating it is connected and the system is ready.
6. Open the QGC application on the operator computer.

7. You can now connect the Xbox controller to the operator computer via Bluetooth. Once connected, it will appear in the top bar of the QGC application.
8. [Continue to Section 4](#) (Operating the System) to connect to the USV computer and run the system.

Note: The Bluetooth Xbox controller pairing only needs to be done once per operator computer. On subsequent sessions, it will normally reconnect automatically once Bluetooth and QGC are both active.

3. The USV Platform

The USV is a custom-built catamaran carrying a Raspberry Pi 4 (“the USV Pi”) as its onboard computer. This Pi is physically and functionally distinct from the ROV’s own Raspberry Pi 4. All USV-specific software, sensors, and actuators described in this manual run on the USV Pi.

3.1 USV GPIO Pin Map

The following table lists every Raspberry Pi 4 physical pin currently assigned to a USV component. Unlisted pins remain free for future use (e.g., I²C devices such as a display, or additional sensors).

Phys. Pin	BCM / Fn	Connected Component	Wire / Signal
1	3.3V	Cable angle encoder	VCC
6	GND	Cable angle encoder	GND
9	GND	HX711 load-cell amplifier	GND
13	GPIO 27	HX711 load-cell amplifier	SCK
15	GPIO 22	HX711 load-cell amplifier	DT
16	GPIO 23	Cable angle encoder	B channel
17	3.3V	HX711 load-cell amplifier	VCC

18	GPIO 24	Cable angle encoder	A channel
29	GPIO 5	Winch motor (iSV57T-130S)	PUL+ (step)
30	GND	Winch motor (iSV57T-130S)	PUL− / DIR− (shorted)
31	GPIO 6	Winch motor (iSV57T-130S)	DIR+ (direction)
32	GPIO 12	Left T200 thruster ESC	PWM signal
34	GND	Left T200 thruster ESC	Signal GND
37	GPIO 26	Right T200 thruster ESC	PWM signal
39	GND	Right T200 thruster ESC	Signal GND

Table A.3: Complete USV Raspberry Pi 4 GPIO assignment

⚠ Warning: The cable angle encoder (AMT112S-V) outputs 5V TTL (Transistor-Transistor Logic) signals natively. On this build it is powered from the Pi's 3.3V rail (pin 1) so that its A/B outputs are also 3.3V — safe for direct connection to Pi GPIO. Do not power this encoder from 5V while its signal lines are wired directly to Pi GPIO pins.

Raspberry Pi 4 - Pin Diagram:

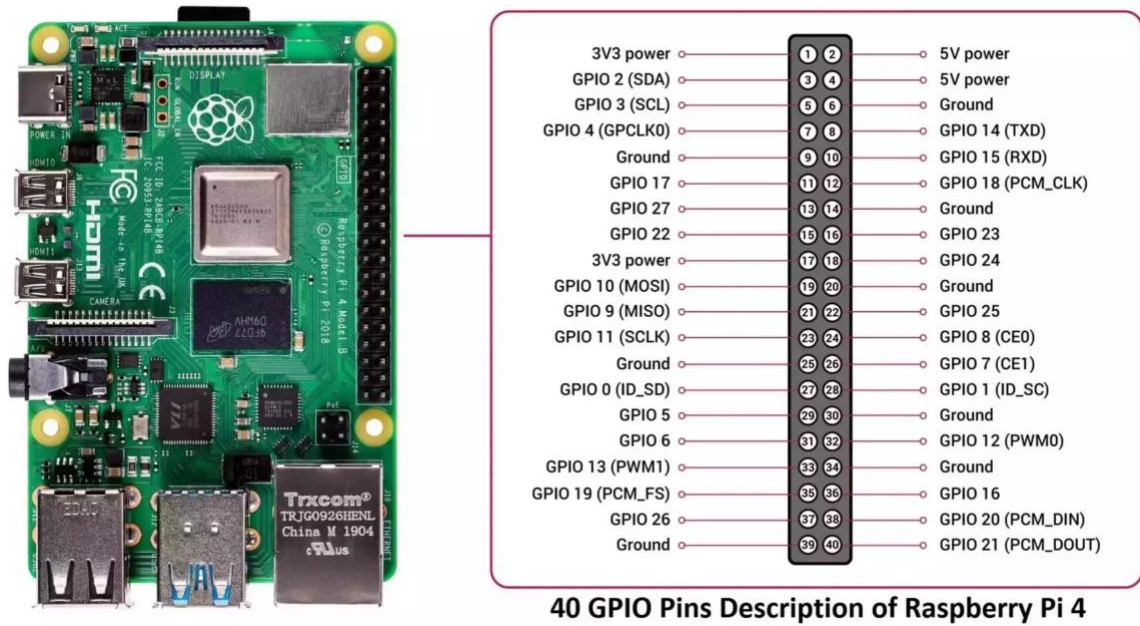


Figure A.1: RPi4 pin diagram

4. Operating the System

This section continues directly from the Power-Up steps in [Section 2.3](#). It tells you exactly what to do, step by step, to connect to the USV and check the system. Follow the steps in order. Each step tells you what to type or click, and what you should see or hear happen. If something does not match what is described, stop and check Section 6 ([Troubleshooting](#)) before continuing.

4.1 Connecting the RPi4 to a Network

Option A - Wired (Ethernet)

1. Plug an Ethernet cable from the RPi4's network port into the switch / router / laptop port.
2. Wait ~10 seconds - the RPi4's onboard LEDs (next to the port) should start blinking.
3. On your laptop, open a terminal and type this exactly and press Enter:

```
ping 192.168.2.3
```

(use the RPi4's known IP - replace if different)

4. If you see replies, you are connected. If not, go to Step 5.
5. If the RPi4 has no fixed IP yet, connect a monitor + keyboard directly to it (or use the SSH (Secure Shell) steps in Option C) and run:

```
ip a
```

to find its current IP address, then ping that instead.

Option B - Wi-Fi (connecting RPi4 to a Wi-Fi network)

1. Connect a monitor, keyboard, and mouse directly to the RPi4 (or SSH in via Ethernet first - see Option C).
2. Open a terminal on the RPi4.
3. Type this exactly and press Enter:

```
sudo raspi-config
```

4. Select **"1 System Options"** → **"S1 Wireless LAN"**.
5. Enter the Wi-Fi network name (SSID - Service Set Identifier) when prompted.

6. Enter the Wi-Fi password when prompted.
7. Select "**Finish**" and reboot if asked:

```
sudo reboot
```

8. After reboot, check the connection:

```
ip a
```

Look for an IP address under **wlan0** - if there is one listed (e.g. **192.168.x.x**), it worked.

Quick alternative (no menu):

Edit the network connection directly by typing this exactly and pressing Enter:

```
sudo nmcli dev wifi connect "NETWORK_NAME" password "PASSWORD"
```

This connects to Wi-Fi in one line - useful for fast field setup.

4.2 Checking the ROV Battery

Before powering anything, check the ROV battery charge. The system must not be operated if the battery is below 50%.

1. Open the ROV's bottom tube by removing its 8 retaining bolts. Set the bolts aside safely.
2. Take out the battery and connect it to its charger using the two dedicated charging cables. Match the connectors carefully.
3. Read the charge level on the charger / battery. Detailed instructions are printed on the battery and in the battery's own user manual — follow those for exact charging steps.
4. If the battery is below 50%, charge it before use. Do not operate the system with a battery below 50%. [How to charge](#)
5. Once charged, reinstall the battery in the bottom tube and refasten all 8 bolts before closing the tube.

⚠ Warning: Watch the battery level continuously during operation. If the battery voltage drops below its minimum safe level, the battery will shut itself off to protect its cells, and it can be very difficult — sometimes impossible — to bring it back into use. Stop and recover the system well before the battery gets that low.

4.3 Connecting to the USV Computer

After the Power-Up steps (Section 2.3) are done and the ROV is connected, connect the operator computer to the USV's onboard computer.

1. On the operator computer, open a terminal window.
2. Type this exactly and press Enter:

```
ssh lar@raspberrypi
```

If you are using a Mac, type this instead:

```
ssh lar@raspberrypi.local
```

3. When asked for a password, type this exactly and press Enter. The letters will not appear on screen as you type — this is normal:

```
qwerty
```

4. Look at the start of the line in the terminal. If it now reads `lar@raspberrypi`, you are connected to the USV's onboard computer. You can now type commands directly into this window - directly on the RPI4 USV.

⚠ Warning: The password shown here (qwerty) is a lab/bench default. Do not rely on it for any system exposed to an untrusted network.

4.4 Checking the ROV Disk Space

The ROV records data while it runs, and these recordings slowly fill its memory (disk). If the disk becomes full, the ROV's software cannot start properly, and the system will not work. Before operating, make sure the ROV has at least 3 GB of free disk space.

1. Connect to the ROV from the operator computer. Open a terminal and type this exactly, then press Enter:

```
ssh pi@192.168.2.2
```

2. When asked for a password, type this exactly and press Enter (the letters will not appear on screen — this is normal):

```
raspberry
```

3. Check the free disk space by typing this exactly and pressing Enter:

```
df -h /
```

4. Look at the line that starts with /dev/root. Read the “Avail” column — this is the free space. It must be at least 3 GB. If it shows less than 3 GB (or “Use%” is near 100%), you must free up space before operating.

Note: To free space, the recorded files (the .mcap recordings in /usr/blueos/userdata/recorder) are usually the cause. Remove old recordings you no longer need — but always check with your team before deleting any recording, in case it is real mission data. Never delete settings, firmware, or configuration files.

⚠ Warning: The ROV recorder keeps writing new files every time the system runs, so the disk fills up again over time. Check the free space before every session, and clear old recordings when needed, so you are not caught with a full disk.

4.5 Freeing Disk Space on the ROV

Connect to the ROV from the operator computer. Open a terminal and type this exactly, then press Enter:

```
ssh pi@192.168.2.2
```

1. When asked for a password, type this exactly and press Enter (the letters will not appear on screen — this is normal):

```
raspberry
```

2. Check the free disk space by typing this exactly and pressing Enter:

```
df -h /
```

3. Look at the line that starts with /dev/root. Read the “Avail” column — this is the free space. If it shows less than 3 GB (or “Use%” is near 100%), continue to free up space.
4. List the biggest recording files (these fill the disk) by typing this exactly and pressing Enter:

```
sudo find /usr/blueos/userdata/recorder -type f -exec du -h {} + 2>/dev/null | sort -h | tail -20
```

5. Delete an old recording you no longer need. Replace the name with a real one from the list above, then press Enter. Repeat for each old file, oldest first:

```
sudo rm /usr/blueos/userdata/recorder/recorder_YYYYMMDD_HHMMSS.mcap
```

6. Check the free space again by typing this exactly and pressing Enter:

```
df -h /
```

7. If you still need more space and BlueOS is running, reclaim unused Docker space by typing this exactly, pressing Enter, then typing y and Enter to confirm:

```
sudo docker system prune -a
```

8. Make sure the “Avail” column now shows at least 3 GB. Only delete recordings (.mcap and video files) your team confirms are not needed - never delete settings, firmware, or configuration files.

4.6 Running the Hardware Test

Run the built-in test program. It checks every sensor and motor on the USV one at a time and tells you if something is wrong.

1. In the terminal connected to the USV, type this exactly and press Enter:

```
sudo pigpiod
```

Nothing will appear on screen after this command — that is correct.

2. Type this exactly and press Enter to move into the project folder:

```
cd /home/lar/USV
```

3. Type this exactly and press Enter to start the test:

```
python3 -m pytest tests/test_comprehensive.py -v -s
```

4. Watch the screen. Each test prints PASSED or FAILED. If a test pauses and says “Press Enter to continue,” do whatever the on-screen instructions say (for example, gently pull the tether), then press Enter on the keyboard. [Test output for example](#)
5. When all tests have run, check that every line says PASSED. If any line says FAILED, go to Section 6 ([Troubleshooting](#)) and find that test’s name.

4.7 ROV Flight Modes

The ROV runs ArduSub firmware, which offers several flight modes. Some modes need the ROV to know its own position underwater, because a GPS does not work underwater, today we supply the GPS signal to the ROV, enabling all modes to be available (Table A.4).

Mode	What it does / why
Manual	Joystick inputs go straight to the thrusters, with no stabilization. Always available — needs no sensors
Acro	Like Manual, but the rate of rotation is stabilized for smoother turns. Needs only the onboard IMU
Stabilize	Holds roll level and keeps heading when you are not turning; vertical movement stays manual. Uses the onboard IMU
Depth Hold	Like Stabilize, plus it holds depth automatically when you are not commanding up/down. Uses the onboard depth sensor (fitted)
Position Hold	Holds the ROV's absolute position. Requirement: Actively relies on the continuous MAVLink GPS-injection stream from the USV to maintain a 3D Fix.
Auto	Runs a pre-planned waypoint mission autonomously. Requirement: The operator must first upload a valid mission profile via QGroundControl How to upload a mission
Guided	Let the operator command the ROV to a specific point on the map Requirement: Actively relies on the continuous MAVLink GPS-injection stream from the USV.

Table A.4: ArduSub flight modes

5. Controller (Xbox) Button Mapping

The ROV flows with an Xbox controller connected to the operator computer through QGC. Each button can be set to do a specific action (for example: arm the motors, switch flight mode, turn the lights up or down, or take a photo). This section shows where each button is, lets you write down what your controller is set to do, and explains how to change a button's action in QGC.

5.1 Controller Layout

The diagram below numbers each control on the Xbox controller. The configuration table in Section 5.2 matches each number to its button name and your assigned action.



Figure A.2: Xbox controller controls (numbers match the No. column in Table A.5)

5.2 Our Button Configuration

Fill in each control's action below. The "No." column matches Figure A.5; the "Shift Function" is the action when the Shift button (default: A) is held.

Button	Function	Shift Function
A	Shift	-
B	Open robotic arm	Manual mode
X	Close robotic arm	Depth hold mode
Y	Stabilize mode	Auto mode
LB (Left Bumper)	Camera tilt up	Mount pan right
RB (Right Bumper)	Camera tilt down	Camera source toggle
LT (Left Trigger)	-	-
RT (Right Trigger)	-	-
D-Pad Up	Gain increment	Trim pitch decrement
D-Pad Down	Gain decrement	Trim pitch increment
D-Pad Left	Light dimmer	Trim roll increment
D-Pad Right	Light brighter	Trim roll decrement
View button	Disarm	-
Menu button	Arm	-
Left Stick (press)	Throttles up-down	-
Right Stick (press)	Throttles left-right	-

Table A.5: Controller button configuration (current setup)

Note: The two sticks normally control movement and are not set in the button list: the left stick moves the ROV forward/back and left/right, and the right stick controls up/down and turning.

The table above is for the buttons (and stick presses), which trigger actions such as arming, mode changes, lights, and camera.

5.3 Changing a Button's Action in QGroundControl

To change what a button does, follow these steps in QGroundControl with the controller connected:

1. Open QGroundControl and make sure the controller is connected (the controller icon shows in the top bar).
2. Open the Vehicle Setup page, then choose Joystick from the sidebar.
3. Go to the Button Assignment area. QGC shows a numbered list of all buttons with their current actions, in two columns: Function and Shift Function.
4. Press the physical button you want to change on the controller. Its number lights up in the list, so you know which row to edit.
5. In that row, pick the action you want from the dropdown in the Function column (for the normal press) or in the Shift Function column (for when the Shift button is held).
6. Repeat for any other buttons. The settings are saved automatically and stay set for next time.

6. Troubleshooting

Find what you see in the left column below, then follow the instructions in the right column.

What you see/hear	What to do
No second sound from the ROV after 2 minutes	Check that all cables are plugged in firmly. Disconnect the ROV battery, wait 10 seconds, then reconnect it and wait 2 minutes again. Check the ROV battery
Terminal says “connected: False”	Type this and press Enter, then try the test again: sudo pigpiod
A thruster does not beep when powered on	Turn off the power supply output immediately. Check that the wire from the Pi to the ESC is firmly connected, then turn the output back on
A power supply screen shows “ALARM” and shuts off	Press the Output button to turn it off completely. Increase the OVP (upper voltage limit) setting on that power supply by 2–3 V, then turn it back on
Tension reading on screen jumps to zero sometimes	This is a known, minor issue. You can continue — it does not stop the system from working correctly
Encoder number does not change when you move the cable by hand	Turn everything off. Check that all 4 small wires on the encoder are firmly plugged into the correct pins, then turn the system back on and try again
Winch motor does not move at all	Turn off the winch power supply. Check that all 3 wires from the Pi to the winch motor are firmly connected, then turn the power back on
No GPS signal / position shown	Move the GPS antenna to a spot with a clear, open view of the sky (not indoors, not under cover). Wait a few minutes

The test program shows a red error message instead of PASSED/FAILED	Make sure you moved into the project folder first: <pre>cd /home/lar/USV</pre> and that you ran the test with python3 (not just pytest): <pre>python3 -m pytest tests/test_comprehensive.py -v -s</pre>
Cannot ping the antennas (10.0.0.10 / 10.0.0.11) or the ROV, even though the link "worked before"	Check the USV Pi's network. Type this and press Enter: <pre>ip a</pre> Confirm the br0 line shows inet 192.168.2.3, and look for an interface with a 10.0.0.x or expected address. If an interface shows DOWN or NO-CARRIER, that cable/link is the problem
Want to see all interfaces and their IP addresses (older style)	Type this and press Enter: <pre>ifconfig</pre> Check that eth0, eth1 (the FXTI), and br0 are listed and have the expected addresses. A missing interface means its cable or USB device is not connected
Cannot reach the ROV (192.168.2.2) but the antennas are fine	Check the routing and which devices answer. Type these and press Enter: <pre>ip route arp -n</pre> If 192.168.2.2 shows "(incomplete)", nothing is answering — the ROV's link (FXTI/tether) is down or BlueOS has not finished booting. Confirm the FXTI Power and Link lights are on, then wait 2–3 minutes
The FXTI (USB-Ethernet to the ROV) does not appear as an interface	Type this and press Enter: <pre>lsusb</pre> Look for "Realtek RTL8152" (the FXTI). If it is missing, re-seat the FXTI's USB cable in the Pi's blue port and check its power

Table A.6: Common problems and what to do about them

[More possible Troubleshooting](#)

Please be aware of: [Safety first](#)

9.3 Technical Manual

TECHNICAL MANUAL

Hovering Autonomous Underwater Vehicle (HAUV) System

ROV • USV • Surface Relay • Tether • Ground Control

Project p-2026-031

Ben-Gurion University of the Negev

Faculty of Engineering Sciences — Dept. of Electrical and Computer Engineering

Students: Guy Hatzir · Maayan Sarig

Supervisor: Prof. Hugo Guterman

Table of Contents

1. System Architecture	61
1.1 The Three Platforms	62
1.2 Communication Links	62
1.3 Network Addresses	63
2. Operating States and Control Flow	64
2.1 System State Machine	64
2.2 Auto-Mode Reactive Loops	64
2.3 Signal Flow - Sensors to Outputs	65
3. Electrical and Power System	66
3.1 Electrical Properties of Components	66
3.2 Power Supplies	66
4. The ROV - BlueROV2	67
4.1 General Characteristics	67
4.2 The Tether	67
4.3 Fathom-X Tether Interface (FXTI)	68
4.4 Power and Battery	69
4.5 Onboard Sensors and Thrusters	69
4.6 Flight Modes	69
5. USV Subsystems and Interfaces	71
5.1 USV Computer Environment	71
5.2 GPIO Pin Map	71
6. Software Architecture and Tracking Algorithm	74
6.1 The Follower (Tracking) Node	74
6.2 RTL Mode	75
6.2.1 Activation Triggers	75
6.2.2 Synchronized Recovery Sequence	76
6.3 Other Nodes	76
6.4 The pigpio Daemon	76
7. Ports and Data Paths	76
7.1 Data Paths Overview	77
7.2 Physical Connection Ports	78
8. Assembly and Power-Up	79

8.1 Component List	79
8.2 System Assembly - Step by Step	80
8.3 Power-Up the System	81
9. Operating the System	82
9.1 Checking the ROV Battery	82
9.2 Charging ROV battery	82
9.3 Connecting to the USV Computer	83
9.4 Checking the ROV Disk Space	84
9.5 Freeing disk space - Step by Step	85
9.6 Running the Hardware Test	85
10. Antenna Network Setup	86
10.1 Antenna Addresses and Roles	86
10.2 Reaching an Antenna's Web Interface	86
10.3 Settings You May Need to Change	87
10.4 After Changing Settings	87
11. Controller Button Mapping	88
11.1 Controller Layout	88
11.2 Button Configuration	89
11.3 Changing a Button's Action in QGroundControl	90
12. Verification Experiments	90
12.1 Angle-Sensor Reading Verification	90
12.2 Thruster / Angle-Sensor Interface Experiment	91
12.3 Tension-Sensor Sampling Experiment	91
12.4 Position Experiment	91
12.5 Extended System Experiment	91
12.6 Test Output	92
13. Mission Planning and Autonomous Execution via QGroundControl	99
13.1 Defining an Autonomous Waypoint Mission (Auto Mode)	99
13.2 Mission Synchronization and Upload	100
13.3 Activating 'Auto' and 'Guided' Modes	102
14. Troubleshooting	103
15. Safety Summary	105

1. System Architecture

The system is composed of three physically separate platforms (Figure B.1 + Table B.1), linked by two communication channels. The operator commands the system from the Mother Ship, the USV relays between the operator and the ROV, and the ROV (Figure B.2) performs the underwater mission.

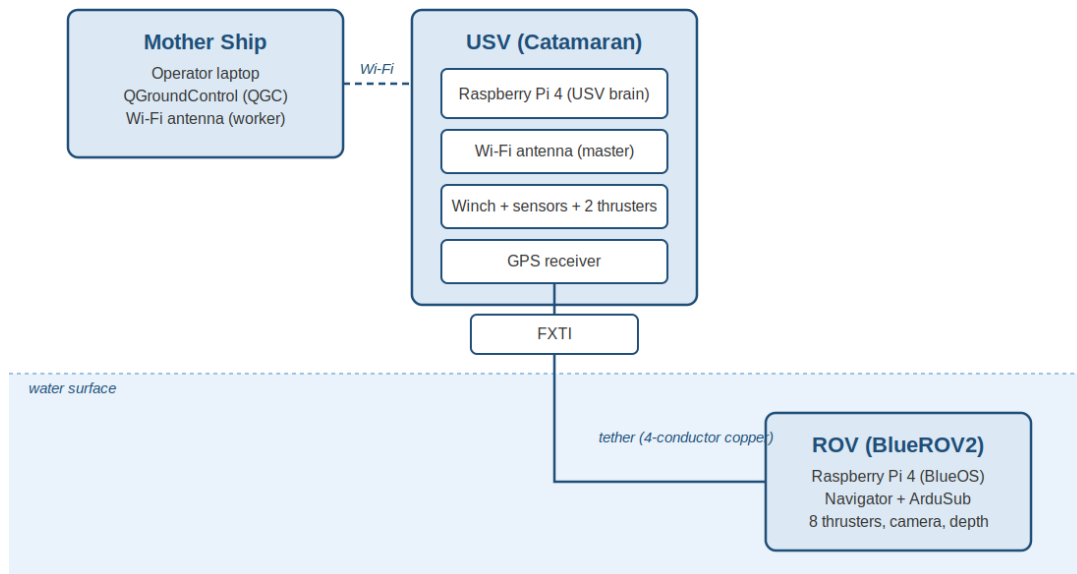


Figure B.1: System architecture: Mother Ship (Operator), USV, and ROV, with the RF surface link and the underwater tether



Figure B.1: ROV

1.1 The Three Platforms

Platform	Role	Key hardware
Mother Ship	Mission planning and operator console	Laptop running QGroundControl, Wi-Fi antenna (worker)
USV (catamaran)	Communication relay and autonomous follower	Raspberry Pi 4, two T200 thrusters, winch, sensors, Wi-Fi (master), GPS
ROV (BlueROV2)	Primary autonomous agent underwater	Raspberry Pi 4, Navigator flight controller, eight thrusters (Heavy configuration), camera, depth sensor

Table B.1: System platforms and their roles

1.2 Communication Links

- Surface link (Mother Ship ↔ USV): long-range Wi-Fi (2.4/5 GHz) between two paired antennas — a master on the USV and a worker on the operator side — carrying MAVLink telemetry, control, and relayed video.
- Underwater link (USV ↔ ROV): a tethered 4-conductor copper cable carrying MAVLink commands down to the ROV and video/telemetry back up, via the Fathom-X Tether Interface (FXTI) on the USV side.

Note: There are two Raspberry Pi 4 units in the system. “The USV Pi” (user lar) runs all USV-specific software described in this manual. “The ROV Pi” runs BlueOS/ArduSub and must not be modified as part of USV work.

1.3 Network Addresses

Device	IP Address
ROV	192.168.2.2
USV	192.168.2.3
Laptop (operator computer) * Modification is needed for the other operator config	192.168.2.150
Antenna - master (USV side)	10.0.0.10
Antenna - worker/slave (operator side)	10.0.0.11

Table B.2: Network IP addresses

2. Operating States and Control Flow

The system runs as a state machine. It boots into Station-Keeping and transitions between Manual, Auto, and RTL based on operator commands and fault conditions. This chapter shows the states, the transitions, and how sensor data flows through the control nodes to the actuators.

2.1 System State Machine

On boot, the USV holds position in Station-Keeping. An operator joystick input enters Manual, and a MAVLink mission-start enters Auto mode. Any major fault - communication loss, ROV fault, tension overload, cable disconnect, signal loss, or battery failure - forces a transition to RTL, in which the ROV ascends to a safe depth while the USV holds tension, then the USV navigates both vehicles back to the launch coordinates before recovery and shutdown.

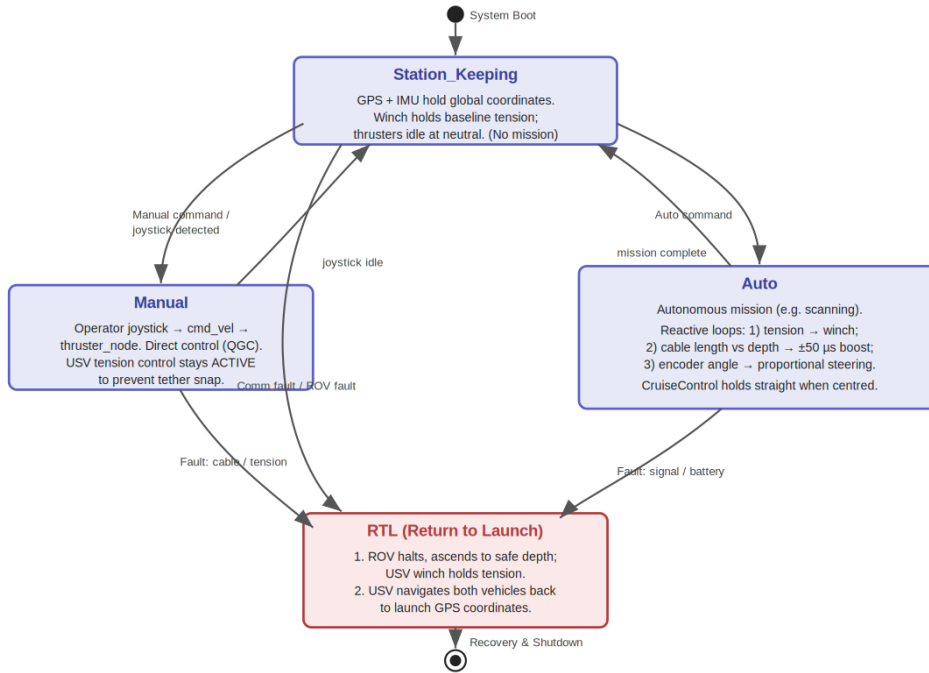


Figure B.3: System state machine: Station-Keeping, Manual, Auto, and the fault-driven RTL state

2.2 Auto-Mode Reactive Loops

In Auto mode, the follower runs three reactive loops in parallel: (1) tension → winch pay-out/reel-in to keep the tether tension in band, (2) cable length versus ROV depth → a forward/back boost of about $\pm 50 \mu\text{s}$ on the thrusters; and (3) encoder angle → proportional steering. The Cruise Control sub-state holds the vessel (USV) straight when the cable angle is

small (within about $\pm 10^\circ$), and the tension is in band, while all three reactive loops remain armed.

2.3 Signal Flow - Sensors to Outputs

The diagram below traces every signal from the ROV's underwater movement, through the USV and ROV sensors, into the ROS 2 control nodes, and out to the physical USV response. It is the single most useful reference for understanding what each reading drives.

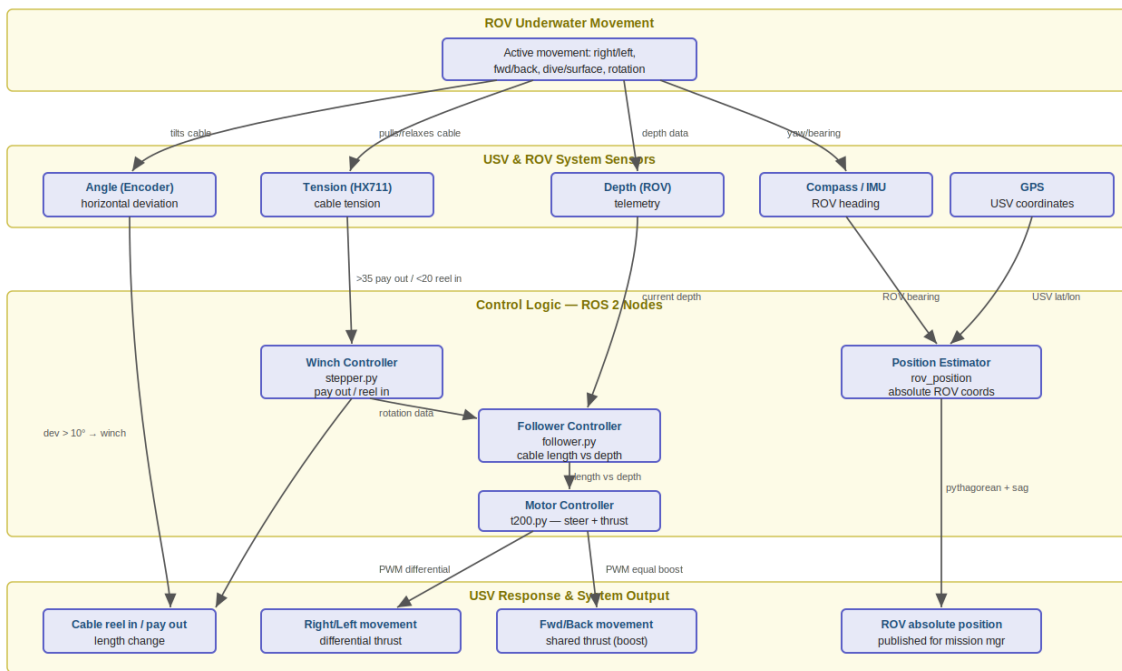


Figure B.4: End-to-end signal flow from ROV movement through the sensors and ROS 2 control nodes to the USV outputs

- ROV tilts the cable sideways → the angle encoder reads horizontal deviation → above 10° , the motor controller steers via differential thrust between the two T200 motors.
- ROV pulls/relaxes the cable → the HX711 reads tension → above 35 kg the winch pays out, below 20 kg it reels in.
- ROV depth (MAVLink) → the follower compares cable length to target depth → too long gives negative thrust, too short gives positive thrust (shared forward/back boost).
- ROV heading and USV GPS → the position estimator computes absolute ROV coordinates (Pythagorean + cable-sag factor + offset) and publishes them for the mission manager.

3. Electrical and Power System

The vessel's autonomy is enabled by a system of sensors and motors controlled by the following algorithm running on a microcontroller - a Raspberry Pi 4 - which holds the tracking algorithm. The controller connects to and receives data from the sensors below and drives three motors (the winch stepper and two identical propulsion thrusters), with additional components including batteries, a transmitter, and an antenna for operator communication.

3.1 Electrical Properties of Components

To choose the power cables and the power board, the electrical properties of each electronic part were computed and matched to a suitable cable by price and weight, per part. The table below lists the electrical properties of each component.

Device	Typ. V	Max V	Typ. A	Max A	Typ. W
Angle sensor	5	—	0.016	—	0.08
Stepper motor	28	50	4.643	6	130
Thrusters	14	20	1.2	32	16.8
Tension sensor	5 (pi gpio)	5.5	0.002		0.01
RPI4	5		0.4 - 1.2		2 - 6

Table B.3: Electrical properties of system components

3.2 Power Supplies

- Thruster bench supply (TDK-Lambda GENH750W): set to 12V, current limit 8 A, OVP 17 V (raised after a regenerative spike on fast reverse tripped a lower OVP), UVL 0 V.
- Winch driver supply (TDK-Lambda JWS150-24/A): 24 V.

⚠ Warning: Fast motor reversal causes regenerative voltage spikes back into the supply. Set the over-voltage protection (OVP) with headroom above the operating voltage, or the supply will trip and shut off mid-operation.

4. The ROV - BlueROV2

The ROV used in this project is a Blue Robotics BlueROV2, a commercially available remotely operated underwater vehicle. This chapter summarizes the official Blue Robotics hardware relevant to operating and maintaining the ROV. For full assembly instructions, always refer to the official Blue Robotics documentation at bluerobotics.com.

4.1 General Characteristics

Property	Value
Type	Remotely Operated Vehicle (ROV)
Displacement	~12 kg
Length	~45 cm
Depth rating	Up to 300 m (Aluminum enclosure, this build); 100 m with the acrylic enclosure option
Propulsion	Eight vectored thrusters (Heavy Configuration)
Power	Lithium-ion battery, typically 16.8V (4S)
Companion computer	Raspberry Pi 4 (“ROV Pi”), running BlueOS
Flight controller	Navigator, running ArduSub firmware

Table B.4: BlueROV2 general characteristics

Click here to see more <https://bluerobotics.com/store/rov/bluerov2/>

4.2 The Tether

The ROV connects to the surface through a tether carrying both power and data. The underwater link is a 4-conductor copper cable. Only one twisted pair (blue and white) is required for core ROV communication and is wired into the Fathom-X Tether Interface (FXTI). Additional conductors are reserved for accessories such as lights, manipulators, or auxiliary sensors.

⚠ Warning: Do not step on the tether, drive over it with a boat propeller, or allow it to contact sharp objects such as rocks or coral. Always manage tether slack actively while operating the ROV.

4.3 Fathom-X Tether Interface (FXTI)

The FXTI is the surface-side box that converts the tether's signal into a standard Ethernet/USB connection usable by a computer or, in this project, by the USV's Raspberry Pi. In this build the FXTI sits on the USV side of the tether, since the USV Pi — not a topside laptop — is the device that talks directly to the ROV.

1. Insert the tether's Binder connector into the FXTI's bulkhead receptacle, aligning the white arrow on the plug with the white line on the receptacle.
2. Rotate the connector clockwise until it locks, with the green lock indicator at the 12 o'clock position.
3. Connect the FXTI to the USV Pi using the supplied USB cable.
4. Power on the FXTI. The Power and Link indicator lights should illuminate, confirming the tether link is active.



Figure B.5: FXTI front



Figure B.6: FXTI back

Click here to see more <https://bluerobotics.com/learn/fathom-x-tether-interface-fxti-usage-guide/>

4.4 Power and Battery

- The ROV battery cable with red heat-shrink is POWER+. The cable with black heat-shrink is POWER. Always verify polarity before connecting.
- Typical battery: a 4S lithium battery, nominal 14.8 V. There is no internal Battery Management System (BMS), so the voltage must be monitored manually.
- Before each deployment, inspect all penetrators, O-rings, and bottle connectors and lubricate with waterproof silicone grease as needed.

⚠ Warning: Do not discharge the 4S battery below 12.8 V (3.2 V per cell). Below this, the cells can become permanently damaged and may be impossible to recover. Monitor the voltage during operation and stop well before this limit.

4.5 Onboard Sensors and Thrusters

- Eight vectored thrusters (Heavy Configuration) provide full 6-DOF control (surge, sway, heave, roll, pitch, yaw) with improved stability and payload over the standard six-thruster configuration.
- A depth sensor (Bar30 or equivalent) provides real-time depth for Depth Hold mode.
- A main camera streams live video up the tether to the operator via the USV relay.
- An IMU provides attitude data used by ArduSub for Stabilize and Depth Hold modes.

4.6 Flight Modes

The ROV runs ArduSub firmware, which offers several flight modes. Some modes need the ROV to know its own position underwater, because a GPS does not work underwater, today we supply the GPS signal into the ROV, enabling all modes to be available.

Mode	What it does / why
Manual	Joystick inputs go straight to the thrusters, with no stabilization. Always available - needs no sensors
Acro	Like Manual, but the rate of rotation is stabilized for smoother turns. Needs only the onboard IMU
Stabilize	Holds roll level and keeps heading when you are not turning; vertical movement stays manual. Uses the onboard IMU
Depth Hold	Like Stabilize, plus it holds depth automatically when you are not commanding up/down. Uses the onboard depth sensor (fitted)
Position Hold	Holds the ROV's absolute position. Requirement: Actively relies on the continuous MAVLink GPS-injection stream from the USV to maintain a 3D Fix
Auto	Runs a pre-planned waypoint mission autonomously. Requirement: The operator must first upload a valid mission profile via QGroundControl How to upload a mission
Guided	Let the operator command the ROV to a specific point on the map. Requirement: Actively relies on the continuous MAVLink GPS-injection stream from the USV

Table B.5: BlueROV2 / ArduSub flight modes

5. USV Subsystems and Interfaces

The USV carries a Raspberry Pi 4 (“the USV Pi”, user lar) running Ubuntu 20.04.6 LTS and ROS 2 Foxy. This chapter documents each sensor and actuator, its wiring to the Pi, and the engineering notes from bring-up. Each component was proven standalone with a Python test script before integration into ROS 2.

5.1 USV Computer Environment

Property	Value
Operating system	Ubuntu 20.04.6 LTS (64-bit, aarch64)
Middleware	ROS 2 Foxy
User account	lar
PWM / servo timing	pigpio (daemon: pigpiod)
Digital I/O, interrupts	RPi.GPIO
Test framework	pytest (run as python3 -m pytest, not bare pytest)

Table B.6: USV Pi software environment

Note: On Ubuntu 20.04, the apt package python-pytest installs the Python 2 version. Always run `python3 -m pytest` so the correct interpreter is used.

5.2 GPIO Pin Map

The following table lists every Raspberry Pi 4 physical pin currently assigned to a USV component. Unlisted pins remain free for future use (for example I²C devices such as a display, or additional sensors).

Phys. pin	BCM / Fn	Connected component	Wire / signal
1	3.3V	Cable angle encoder	VCC
6	GND	Cable angle encoder	GND
9	GND	HX711 load-cell amplifier	GND
13	GPIO 27	HX711 load-cell amplifier	SCK
15	GPIO 22	HX711 load-cell amplifier	DT
16	GPIO 23	Cable angle encoder	B channel
17	3.3V	HX711 load-cell amplifier	VCC
18	GPIO 24	Cable angle encoder	A channel
29	GPIO 5	Winch motor (iSV57T-130S)	PUL+ (step)
30	GND	Winch motor (iSV57T-130S)	PUL- / DIR- (shorted)
31	GPIO 6	Winch motor (iSV57T-130S)	DIR+ (direction)
32	GPIO 12	Left T200 thruster ESC	PWM signal
34	GND	Left T200 thruster ESC	Signal GND
37	GPIO 26	Right T200 thruster ESC	PWM signal
39	GND	Right T200 thruster ESC	Signal GND

Table B.7: USV Raspberry Pi 4 GPIO

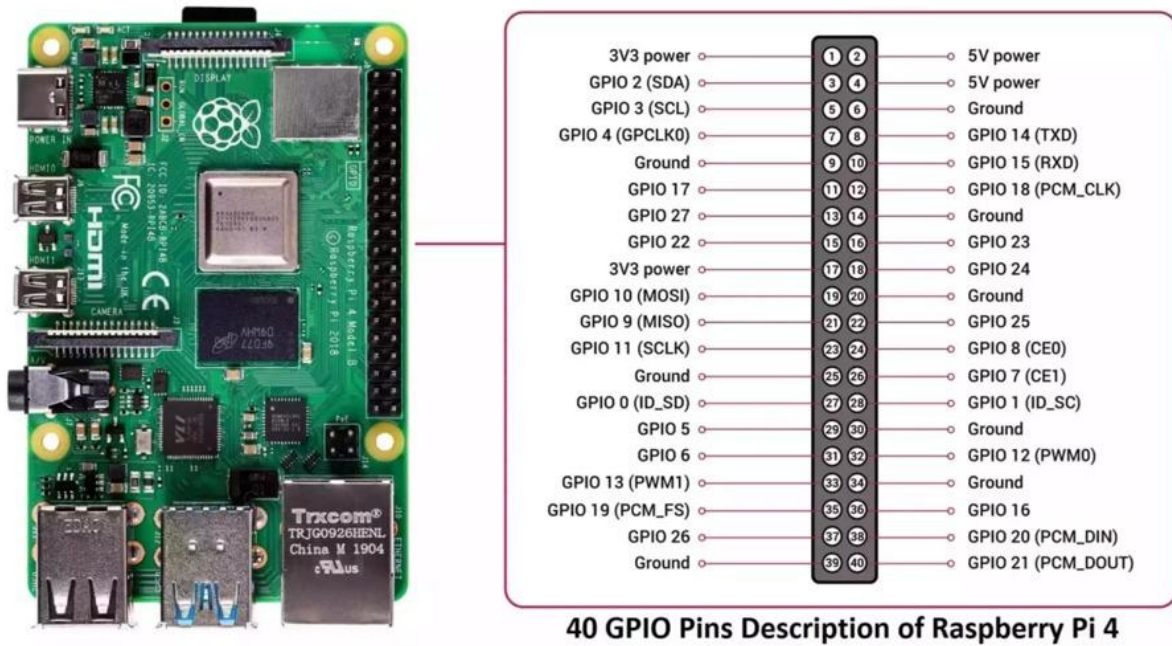


Figure B.7: RPI4 pin diagram

⚠ Warning: The cable angle encoder (AMT112S-V) outputs 5 V TTL signals natively. On this build, it is powered from the Pi's 3.3 V rail (pin 1) so that its A/B outputs are also 3.3 V — safe for direct connection to Pi GPIO. Do not power this encoder from 5 V while its signal lines are wired directly to Pi GPIO pins.

6. Software Architecture and Tracking Algorithm

All USV software runs as ROS 2 Foxy nodes on the USV Pi. Each sensor and actuator is a node; nodes communicate over topics. Sensor nodes publish their readings, the follower node subscribes to them and computes commands, and the actuator nodes subscribe to those commands and drive the hardware. Each node runs in its own thread.

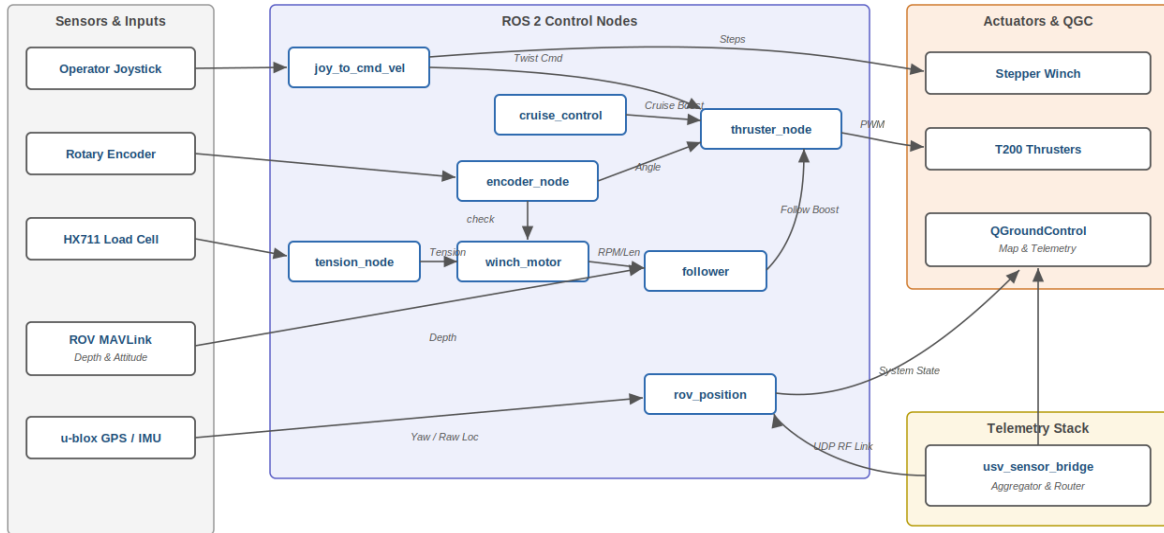


Figure B.8: ROS 2 control-node graph: sensors and inputs, control nodes, the telemetry stack, and the actuators / QGC

6.1 The Follower (Tracking) Node

The USV's heading correction is driven by the cable angle from the encoder. The control logic uses three response zones, allowing gentle correction for small deviations and strong correction for large ones.

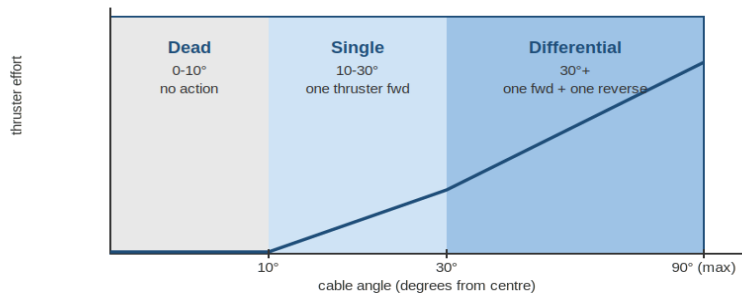


Figure B.9: Follower response zones: dead zone, single-thruster zone, and differential zone as a function of cable angle

Zone	Angle range	Thruster behaviour
Dead zone	0–10°	Both thrusters at neutral (1500 μ s) - no correction
Single thruster	10–30°	Only the thruster on the side the cable pulls toward is driven forward, proportional to the angle; the other stays neutral
Differential	30° and above	The thruster on the cable's side drives forward while the opposite thruster drives in reverse, both proportional to angle, for maximum turning authority

Table B.8: Follower control response zones

Within each zone the commanded pulse width scales linearly with the magnitude of the cable angle, between a minimum engagement threshold just above the neutral dead-band and a maximum correction clamped at a configurable angle (e.g. 90° $\rightarrow$ 1700 μ s). This gives a smooth, proportional response rather than abrupt on/off correction. The sign convention was corrected after physical observation: cable to the right gives a negative angle and drives the right thruster forward; cable to the left gives a positive angle and drives the left thruster forward.

Note: Beyond heading correction, the full follower logic also applies a forward “boost” on top of the correction, so the USV continuously advances to stay above the ROV rather than only reacting to heading error.

6.2 RTL Mode

The RTL Failsafe Mode serves as a critical autonomous failsafe mechanism.

Engineered to protect the integrated USV-ROV system during severe operational faults.

6.2.1 Activation Triggers

This mode is automatically initiated by the following critical emergencies:

- Prolonged RF communication timeouts (30 sec).
- Extreme tether tension overload (exceeding 135 kg).

- Critical power loss.
- Detection of internal water leakage within the ROV.

6.2.2 Synchronized Recovery Sequence

Upon activation, the state machine executes the following steps:

- ROV Action - Immediately aborts its current mission and ascends to a predefined safe depth.
- Winch Action - Holds a baseline tether tension to prevent cable entanglement or snapping.
- USV Action - Acts as the primary surface navigator, utilizing its onboard GPS to autonomously drive both platforms back to their original launch coordinates.
- System Shutdown - Initiates a safe system shutdown once the platforms have arrived at the launch coordinates.

6.3 Other Nodes

- IMU node - publishes acceleration and angular velocity (6-DOF) for attitude awareness and future sensor fusion.
- GPS node - reads the receiver over USB using the UBX binary protocol (u-blox), decoded with the pyubx2 library, and publishes the USV fix. The fix is injected into the QGroundControl MAVLink stream so the operator sees the USV position on the map.
- Display/logger node - records system status and sensor values for later analysis.

6.4 The pigpio Daemon

Stable 50 Hz ESC signals require precise hardware timing that pure-Python loops cannot guarantee. The pigpio daemon (pigpiod) generates hardware-timed PWM for the thrusters. It must be running before the thruster node starts; it is started with `sudo pigpiod` and is configured to auto-start at boot via a crontab `@reboot` entry.

7. Ports and Data Paths

This chapter is the single reference for where each kind of data travels in the system: which device originates it, what protocol and port carry it, and where it ends up. Use it when diagnosing a missing video feed, a lost GPS fix, or a broken telemetry link.

7.1 Data Paths Overview

Data	Source → Destination	Protocol / port	Path
Video	ROV camera → operator (QGC)	UDP video stream, port 5600	ROV → tether → FXTI → USV Pi → Wi-Fi → QGC
ROV control / telemetry	QGC ↔ ROV	MAVLink over UDP, port 14550	QGC ↔ Wi-Fi ↔ USV Pi ↔ FXTI ↔ tether ↔ ROV
ROV depth & attitude	ROV MAVLink → USV nodes	MAVLink (UDP)	ROV → tether → USV Pi (follower, rov_position)
USV GPS fix	u-blox GPS → USV Pi → QGC	UBX over USB; injected to MAVLink	GPS → USB → gps node → usv_sensor_bridge → Wi-Fi → QGC
USV sensor telemetry	USV nodes → QGC	UDP RF link (usv_sensor_bridge)	ROS 2 topics → usv_sensor_bridge → Wi-Fi → QGC
Encoder / tension / IMU	USV sensors → USV Pi	GPIO / I ² C (local)	Sensor → GPIO pins → ROS 2 node (local to USV Pi)
Thruster / winch commands	USV Pi → actuators	PWM (pigpio) / pulse+dir	thruster_node, winch_motor → GPIO → ESC / driver
SSH (maintenance)	Operator → USV Pi / ROV Pi	SSH, TCP (Transmission Control Protocol) port 22	Laptop → Wi-Fi/Ethernet → raspberrypi (USV) or 192.168.2.2 (ROV)
BlueOS web UI (User Interface)	Operator → ROV Pi	HTTP (HyperText Transfer Protocol) , TCP port 80 (and :8080)	Browser → 192.168.2.2 → BlueOS

Table B.9: System data paths, protocols, and ports

Note: Port numbers follow the BlueROV2 / ArduSub defaults (MAVLink 14550, video UDP 5600). If a feed is missing, confirm the relevant port is not blocked and that the antenna link (Chapter 12) is up before looking deeper.

7.2 Physical Connection Ports

Device	Port used	Connects to
USV Pi	USB-A (blue, USB 3.0)	FXTI (USB-B)
USV Pi	USB-A (any free)	u-blox GPS receiver
USV Pi	Ethernet (LAN)	PoE injector → master antenna
USV Pi	Type-c Power	Power source
Operator laptop	Ethernet (LAN)	PoE injector → worker antenna
USV Pi	GPIO header	Thrusters, winch, encoder, HX711
ROV	Tether (Binder)	FXTI bulkhead

Table B.10: Physical connection ports



Figure B.10: ROV components and connection

8. Assembly and Power-Up

This chapter is the step-by-step procedure for assembling the system and powering it up from a cold start. Component lists are grouped by which side of the tether they belong to.

8.1 Component List

ROV side

- ROV - Blue Robotics BlueROV2
- Tether spool
- Short tether cable
- USB Type-B to USB Type-A cable

USV side

- USV platform - catamaran chassis
- Raspberry Pi 4 (USV Pi)
- Mounted tether spool + driver (winch)
- Thrusters - T200 (×2)
- Tension gauge - HX711 + load cell

- Angle encoder
- GPS antenna and receiver
- Antenna + PoE ($\times 2$)
- FXTI (Fathom-X Tether Interface)

External

- Thruster power source (bench power supply)
- Winch driver power source (bench power supply)
- Operator computer

8.2 System Assembly - Step by Step

1. Connect the ROV battery to power the ROV - bottom tube to upper tube (this powers up the ROV).
2. Connect the ROV upper tube to the tether spool.
3. Connect the tether spool, via the short tether cable, to the FXTI.
4. Connect the FXTI to the USV Raspberry Pi 4 using the USB Type-B to USB Type-A cable (into the RPi4's blue USB port).
5. Connect the GPS antenna and receiver to one of the USV Raspberry Pi 4's USB ports (any free USB-A port).
6. Connect the thrusters, winch/spool driver, tension gauge, and angle encoder wires to the RPi4 pins, using the GPIO pin map (Table 7.2).
7. Connect the USV RPi4 power supply to mains power (220V).
8. Connect the first PoE unit to the USV RPi4 (LAN port) and to its antenna (PoE port), and connect the PoE unit to mains power (220 V).
9. Connect the second PoE unit to the operator computer (LAN port) and to its antenna (PoE port), and connect the PoE unit to mains power (220 V).
10. Connect the USV thrusters to their power supply (12 V, 5 A; OVP upper limit 14 V; UVL lower limit 0 V).
11. Connect the spool/winch driver to its power supply (24 V).

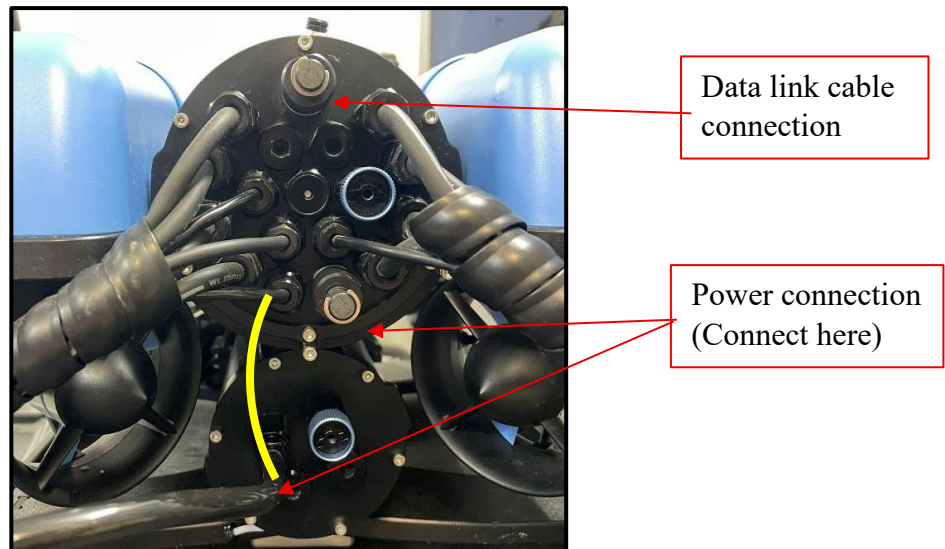


Figure B.11: Network and Power - ROV

8.3 Power-Up the System

This sequence starts its own numbering and is performed after the system is fully assembled.

1. Connect the ROV battery to power the ROV - bottom tube to upper tube. You will hear the internal ROV Raspberry Pi 4 turn on.
2. Click “Output” on both bench power supplies to let power flow to the winch and the thrusters.
3. Power up the two PoE units. On each antenna’s antenna you will see its signal indicator fill the full color bar (yellow to red); the PoE injector itself only shows a single white power light.
4. Open the QGC application on the operator computer.
5. If everything is properly connected, wait about 2 minutes. You will hear a different sound from the ROV, indicating it is connected and the system is ready.
6. Connect the Xbox controller to the operator computer via Bluetooth. Once connected, it will appear in the top bar of the QGC application.
7. Continue to Chapter 9 (Operating the System) to connect to the USV computer and run the system.

Note: The Bluetooth Xbox controller pairing only needs to be done once per operator computer; on subsequent sessions it will normally reconnect automatically once Bluetooth and QGC are active.

9. Operating the System

This chapter continues directly from the Power-Up steps in Chapter 8. It is the operator procedure: connect to the USV, check the system, and run it. Each step states exactly what to type or click and what to expect.

9.1 Checking the ROV Battery

Before powering anything, check the ROV battery charge. The system must not be operated if the battery is below 50%.

1. Open the ROV's bottom tube by removing its 8 retaining bolts. Set the bolts aside safely.
2. Take out the battery and connect it to its charger using the two dedicated charging cables. Match the connectors carefully. [How to charge](#)
3. Read the charge level on the charger / battery. Detailed instructions are printed on the battery and in the battery's own user manual — follow those for exact charging steps.
4. If the battery is below 50%, charge it before use. Do not operate the system with a battery below 50%.
5. Once charged, reinstall the battery in the bottom tube and refasten all 8 bolts before closing the tube.

⚠ Warning: Watch the battery level continuously during operation. If the battery voltage drops below its minimum safe level, the battery will shut itself off to protect its cells, and it can be very difficult - sometimes impossible - to bring it back into use. Stop and recover the system well before the battery gets that low.

9.2 Charging ROV battery

Proper charging of the ROV's 4S Lithium Polymer (LiPo) battery is critical for safety and system longevity. For a comprehensive visual guide, please refer to the official Blue Robotics operation manual: <https://bluerobotics.com/learn/bluerov2-operation/>.

Safety First: Always place the battery entirely inside a fire-retardant LiPo safe charging bag before applying current to minimize fire risks.

Dual-Connection: You must connect both the main discharge lead (XT90) and the white multi-pin balance lead to the charger. The balance lead is vital, as it allows the charger to monitor and equalize the voltage across all four individual cells safely.

Charger Configuration: Set the smart charger to "LiPo BALANCE CHG" mode. Ensure the parameters match the 4S configuration (14.8V nominal) and set an appropriate charging current (typically 1C).

Execution & Monitoring: Initiate the charge cycle. The charger will balance the cells and automatically stop when they reach maximum capacity (4.2V per cell / 16.8V total). Never leave a charging LiPo battery unattended.



Figure B.12: ROV battery charger

9.3 Connecting to the USV Computer

You must be connected to the USV Pi before checking the ROV. Connect the operator computer to the USV's onboard computer first.

1. On the operator computer, open a terminal window.
2. Type this exactly and press Enter:

```
ssh lar@raspberrypi
```

If you are using a Mac, type this instead:

```
ssh lar@raspberrypi.local
```

3. When asked for a password, type this exactly and press Enter. The letters will not appear on screen as you type - this is normal:

```
qwerty
```

4. Look at the start of the line in the terminal. If it now reads `lar@raspberrypi`, you are connected to the USV's onboard computer.

⚠ Warning: The password shown here (qwerty) is a lab/bench default. Do not rely on it for any system exposed to an untrusted network.

Click here to see more <https://bluerobotics.com/learn/bluerov2-software-setup/>

9.4 Checking the ROV Disk Space

The ROV records data while it runs, and these recordings slowly fill its disk. If the disk becomes full, the ROV's software cannot start properly and the system will not work. Before operating, make sure the ROV has at least 3 GB of free disk space.

1. Connect to the ROV from the operator computer. In a terminal type this exactly, then press Enter:

```
ssh pi@192.168.2.2
```

2. When asked for a password, type this exactly and press Enter:

```
raspberry
```

3. Check the free disk space by typing this exactly and pressing Enter:

```
df -h /
```

4. Look at the line that starts with `/dev/root`. Read the "Avail" column — it must be at least 3 GB. If it shows less than 3 GB (or "Use%" is near 100%), free up space before operating.

Note: To free space, the recorded .mcap files in `/usr/blueos/userdata/recorder` are usually the cause. Remove old recordings you no longer need — but always check with your team before deleting any recording, in case it is real mission data. Never delete settings, firmware, or configuration files.

9.5 Freeing disk space - Step by Step

If the ROV disk is full or nearly full, follow these steps. They identify what is using the space before deleting anything, the same careful approach used during bring-up.

1. Make sure you are connected to the ROV (Section 11.3, steps for `ssh pi@192.168.2.2`). Then find the biggest top-level directories:

```
sudo du -h --max-depth=1 / 2>/dev/null | sort -h | tail -15
```

2. The recordings normally dominate. List the recorder folder and its largest files (these are the .mcap recordings and video):

```
sudo find /usr/blueos/userdata/recorder -type f -exec du -h {} + 2>/dev/null | sort -h | tail -20
```

3. Delete specific old recordings you and your team agree are not needed (replace the filename with a real one from the list). Prefer deleting the oldest sessions first:

```
sudo rm /usr/blueos/userdata/recorder/recorder_YYYYMMDD_HHMMSS.mcap
```

4. Check the free space again. You want at least 3 GB free (the “Avail” column):

```
df -h /
```

5. If Docker/BlueOS needs space reclaimed and the BlueOS containers are running, you can prune unused images. Only do this once BlueOS is up, so images in use are kept:

```
sudo docker system prune -a
```

⚠ Warning: Never delete the settings, firmware, or configuration folders in /usr/blueos/userdata. Only remove recordings (.mcap and video files) that your team confirms are disposable. The recorder refills every session, so check free space before every session.

Note: If older commands such as `journalctl --vacuum-size` free nothing, the space is being used by recordings rather than logs — go straight to the recorder folder using the steps above.

9.6 Running the Hardware Test

Before any mission, run the built-in test program. It checks every sensor and motor on the USV one at a time and reports if something is wrong.

1. In the terminal connected to the USV, type this exactly and press Enter:

```
sudo pigpiod
```

Nothing will appear on screen after this command - that is correct.

2. Type this exactly and press Enter to move into the project folder:

```
cd /home/lar/USV
```

3. Type this exactly and press Enter to start the test:

```
python3 -m pytest tests/test_comprehensive.py -v -s
```

4. Watch the screen. Each test prints PASSED or FAILED. If a test pauses and says “Press Enter to continue,” do what the on-screen instructions say (for example, gently pull the tether), then press Enter. [Test output for example](#)
5. When all tests have run, check that every line says PASSED. If any line says FAILED, go to Chapter 16 (Troubleshooting).

10. Antenna Network Setup

The surface Wi-Fi link uses a pair of antennas: a master on the USV/ROV side and a worker (slave) on the operator side. Each Antenna has its own management IP and a web interface for changing its settings. This chapter explains how to reach each antenna and what to change. Where a value is specific to your hardware model, a blank is left for you to fill in.

10.1 Antenna Addresses and Roles

Radio	IP address	Role
Antenna - master	10.0.0.10	USV / ROV side, defines the link and channel
Antenna - worker / slave	10.0.0.11	Operator side, connects to the master

Table B.11: Antenna addresses and roles

10.2 Reaching an Antenna's Web Interface

1. Connect the operator computer to the antenna's network (through its PoE injector and Ethernet, as in the assembly steps).
2. Set the computer's Ethernet adapter to a static IP in the same range, for example 10.0.0.20, mask 255.255.255.0, so it can reach the antennas.

3. Open a web browser and go to the antenna's address — the master at:

http://10.0.0.10

or the worker/slave at:

http://10.0.0.11

4. Log in with the antenna's admin credentials. Username: _____ Password: _____ (fill in your unit's credentials; do not store them in this manual if it will be shared).

10.3 Settings You May Need to Change

1. Wireless mode / role - confirm the master antenna is set as Access Point / Master and the worker as Station / Slave. Swapping these breaks the link.
2. SSID - the link name; must match between the two antennas.
Current SSID: _____
3. Channel / frequency - choose a clear channel to avoid interference; both antennas follow the _____ master.
Current channel: _____
4. Channel width - a narrower width (e.g. 20 MHz) is more robust over distance, a wider width gives more throughput at a shorter range.
5. Security / encryption key - must match on both antennas. Store the key securely, not in a shared document.
6. IP address - keep the master at 10.0.0.11 and the worker at 10.0.0.10 unless you also update Table 2.2 and every reference to them.
7. Output power / distance setting - set according to the operating range and local regulations.

10.4 After Changing Settings

1. Apply / save the changes in the antenna's web interface; the antenna may reboot.
2. Make the matching change on the other antenna if the setting must match (SSID, channel, security key).
3. Confirm the link comes back up: the antenna's signal indicator should fill its color bar (yellow to red), and you should be able to reach both antennas again in the browser.

4. Reconnect QGroundControl and confirm video and telemetry return (see the data paths in Table 11).
5. Set the computer's Ethernet adapter back to its normal setting for operation.

⚠ Warning: Change one antenna at a time and keep a record of the original settings. If both antennas are misconfigured at once, you can lose access to both and may need a physical reset to recover.

Note: The exact menu names depend on the antenna model (for example Ubiquiti airOS or a similar CPE (Customer-Premises Equipment)). The steps above are the general procedure; fill in the model-specific menu names and credentials for your hardware.

11. Controller Button Mapping

The ROV is flown with an Xbox controller connected to the operator computer through QGC. Each button can be set to a specific action (arm the motors, switch flight mode, adjust the lights, take a photo). This chapter shows where each control is, records what each is set to, and explains how to change a button's action in QGC.

11.1 Controller Layout

The diagram below numbers each control on the Xbox controller. The configuration table in Section 13.2 matches each number to its button name and your assigned action.



Figure B.13: Xbox controller controls (numbers match the No. column in Table B.12)

11.2 Button Configuration

Fill in each control's action below. The “No.” column matches Figure 13; the “Shift Function” is the action when the Shift button (default: A) is held.

Button	Function	Shift Function
A	Shift	-
B	Open robotic arm	Manual mode
X	Close robotic arm	Depth hold mode
Y	Stabilize mode	Auto mode
LB (Left Bumper)	Camera tilt up	Mount pan right
RB (Right Bumper)	Camera tilt down	Camera source toggle
LT (Left Trigger)	-	-
RT (Right Trigger)	-	-
D-Pad Up	Gain increment	Trim pitch decrement
D-Pad Down	Gain decrement	Trim pitch increment
D-Pad Left	Light dimmer	Trim roll increment
D-Pad Right	Light brighter	Trim roll decrement
View button	Disarm	-
Menu button	Arm	-

Table B.12: Controller button configuration

11.3 Changing a Button's Action in QGroundControl

1. Open QGroundControl and make sure the controller is connected (the controller icon shows in the top bar).
2. Open the Vehicle Setup page, then choose Joystick from the sidebar.
3. Go to the Button Assignment area. QGC shows a numbered list of all buttons with their current actions, in two columns: Function and Shift Function.
4. Press the physical button you want to change. Its number lights up in the list, so you know which row to edit.
5. Pick the action from the dropdown in the Function column (normal press) or the Shift Function column (when the Shift button is held).
6. Repeat for any other buttons. The settings are saved automatically.

Note: The default Shift button is the A button. A button's Shift Function is what happens when you hold Shift and press that button — letting each button do two things.

12. Verification Experiments

To validate the algorithm and the interfaces, several experiments were performed. This chapter details the experiments and their conclusions.

12.1 Angle-Sensor Reading Verification

On the first installation of the sensor, an experiment found the conversion ratio between the measured voltage value and the physical angle. A shaft of suitable diameter was passed through the sensor, and a start point (zero angle) was marked on it. The experiment showed the encoder default conversion value was not correct, so a correction coefficient was applied, determined from the experiment results; after the calculation, the angle value was published on the topic. A follow-up experiment verified the correction coefficient and whether further correction was needed; the shaft was held against a surface to prevent deviations. The sensor defines its position at power-on as the zero angle; the experiment started at 90° relative to that reference, and all measurements were taken relative to 90°. A return to the zero point was performed to see the effect of movements on the initial zero. Some measurement error arose from the experiment method itself - the printed angle gauge, eye-based reading, and the round (not pointed) screw profile.

12.2 Thruster / Angle-Sensor Interface Experiment

The second part of the experiment proved the thrusters respond according to the angle, to correct the rotation angle above a preset threshold (10°). The angle-sensor readings were logged over the experiment time and confirmed that the follower drives the thrusters proportionally once the cable angle exceeds the dead-band, validating the three-zone control design in software before any in-water trial.

12.3 Tension-Sensor Sampling Experiment

This experiment examined the sampling behaviour of the tension sensor (load cell + HX711). It confirmed the calibration/tare procedure and characterised the intermittent zero-drop behaviour caused by marginal jumper connections, motivating the in-software averaging/Kalman filtering that smooths occasional bad readings.

12.4 Position Experiment

A position experiment verified the GPS interface and the injection of the USV fix into the QGroundControl MAVLink stream, so the operator can see the USV position on the same map as the ROV. GPS requires sky visibility; indoor tests show no fix, which is expected.

12.5 Extended System Experiment

A broad system experiment exercised the integrated node graph end-to-end: encoder → follower → thrusters in a closed loop, with the tension and winch subsystems active. The closed-loop follower demo worked cleanly, with the thrusters responding proportionally to the cable angle and the sign convention confirmed by physical observation. This experiment is the basis for the pre-deployment hardware test ([Section 9.6](#) , [Section 12.6](#)).

12.6 Test Output

USV TEST BENCH - System Bringup

Starting comprehensive system tests... Each test will pause for approval before continuing.

```
=====
test                session                starts
=====
Platform: linux -- Python 3.8.10, pytest-4.6.9, py-
1.8.1, pluggy-0.13.0 -- /usr/bin/python3 Cachedir: .pytest_cache Rootdir: /home/lar/USV,
inifile: pytest.ini Plugins: ament-flake8-0.9.8, ament-pep257-0.9.8, launch-testing-ros-0.11.7,
ament-xmlint-0.9.8, ament-copyright-0.9.8, launch-testing-0.10.10, ament-lint-0.9.8, colcon-
core-0.20.1, cov-2.8.1 Collected: 23 items
```

- tests/test_comprehensive.py::test_pigpiod_alive **PASSED**
- tests/test_comprehensive.py::test_gpio_pins_output **PASSED**
- tests/test_comprehensive.py::test_steering_formula **PASSED**
- tests/test_comprehensive.py::test_cruise_logic **PASSED**
- tests/test_comprehensive.py::test_rov_position_geometry **PASSED**
- tests/test_comprehensive.py::test_station_keeping_math **PASSED**
- tests/test_comprehensive.py::test_rtl_threshold_sanity **PASSED**

tests/test_comprehensive.py::test_tension_reads

[1/8] TENSION SENSOR (HX711)

```
=====
Calibrating...
Calibration complete. Tared value: 1.20 ✓ PASS - Tension: 0.04 kg Press Enter to continue...
PASSED
```

tests/test_comprehensive.py::test_encoder_reads

[2/8] ENCODER (AMT112S-V)

===== ✓ **PASS** - Angle:

0.00° (raw position: 0 ticks) **PASSED**

tests/test_comprehensive.py::test_gps_connect

[3/8] GPS RECEIVER (u-blox)

===== ✓ **PASS** - Received

484 bytes in 5 s (need > 10) **PASSED**

tests/test_comprehensive.py::test_thrusters_pwm

[4/8] THRUSTERS (T200 PWM)

===== ✓ **PASS** - NEUTRAL:

R=1500 μ s L=1500 μ s ✓ **PASS** - FORWARD: R=1600 μ s L=1600 μ s ✓ **PASS** - REVERSE:

R=1400 μ s L=1400 μ s **PASSED**

tests/test_comprehensive.py::test_stepper_steps

[5/8] STEPPER MOTOR (Winch)

===== ✓ **PASS** - DIR

FORWARD: expected=0, got=0 ✓ **PASS** - DIR BACKWARD: expected=1, got=1 ✓ **PASS** -

Step pulse generated (no exception) **PASSED**

tests/test_comprehensive.py::test_tension_stepper_interaction

[6/8] TENSION → STEPPER Interaction

===== Calibrating...

Calibration complete. Tared value: 1.23 Pull and release the cable during the next 10 seconds...

- [1/10] tension = 0.02 kg
- [2/10] tension = 0.03 kg
- [3/10] tension = 3.90 kg

- [4/10] tension = 80.87 kg
- [5/10] tension = 60.97 kg
- [6/10] tension = 67.81 kg
- [7/10] tension = 40.90 kg
- [8/10] tension = 1.54 kg
- [9/10] tension = 1.45 kg
- [10/10] tension = 1.49 kg

✓ **PASS** - Variation: 0.02 → 80.87 kg *Press Enter to continue...* **PASSED**

tests/test_comprehensive.py::test_encoder_thrusters_proportional Arming ESC (neutral)...
[INFO] [1782225678.785224273] [thruster_node]: Thruster node ready. **PASSED**

tests/test_comprehensive.py::test_cruise_activation_closed_loop [INFO]
[1782225679.575453143] [cruise_control]: Cruise control ready. Active when $|\text{angle}| < 15^\circ$
(steering dead zone) and $20 < \text{tension} < 35$ kg. Boost=30 μs . ✓ center + nominal → **CRUISE**
ON: boost=30 μs ✓ angle > dead zone → **CRUISE OFF**: boost=0 μs ✓ low tension →
CRUISE OFF: boost=0 μs ✓ high tension → **CRUISE OFF**: boost=0 μs ✓ back to nominal
→ **CRUISE RESUMES**: boost=30 μs **PASSED**

tests/test_comprehensive.py::test_overtension_stepper_response DIR_PIN state after
tension=40 kg: 1 (1=reel-out) **PASSED**

tests/test_comprehensive.py::test_thruster_mode_switching Arming ESC (neutral)... ✓ Mode
gate verified: SK=ignored, MANUAL=active, RTL=ignored **PASSED**

tests/test_comprehensive.py::test_rov_connection Connecting to vehicle on
udpin:0.0.0.0:14551... Connected! Heartbeat received from system 1 component 0 Connected:
sysid=1 compid=0 **PASSED**

tests/test_comprehensive.py::test_rov_arm_disarm Sent mode change to MANUAL Sent arm
command **PASSED**

tests/test_comprehensive.py::test_rov_mode_transitions Sent mode change to MANUAL ✓
MANUAL (custom_mode=19) Sent mode change to STABILIZE ✓ STABILIZE

(custom_mode=0) *SKIP: mode 'DEPTH_HOLD' not in firmware mode map (ArduSub version may differ)* Sent mode change to MANUAL ✓ MANUAL (custom_mode=19) **PASSED**

tests/test_comprehensive.py::test_rov_attitude_stream Requested stream 10 at 4 Hz (start_stop=1) ✓ 3 ATTITUDE messages received | last yaw = -126.9° **PASSED**

tests/test_comprehensive.py::test_visual_demo

=====

[9/9] VISUAL DEMO (Thrusters + Winch)

===== ⚠ *Make sure thrusters are clamped and winch cable is free! Press Enter to start visual demo (Ctrl+C to abort)...*

- [ARM (9 s)] R=1500 μ s L=1500 μ s (9 s)
- [GENTLE FORWARD] R=1600 μ s L=1600 μ s (7 s)
- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [FAST FORWARD] R=1700 μ s L=1700 μ s (7 s)
- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [WINCH] REEL IN slow freq=800 Hz (12 s)
- [GENTLE REVERSE] R=1400 μ s L=1400 μ s (7 s)
- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [FAST REVERSE] R=1300 μ s L=1300 μ s (7 s)
- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [WINCH] REEL OUT slow freq=800 Hz (12 s)
- [HARD TURN RIGHT] R=1400 μ s L=1600 μ s (7 s)
- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [HARD TURN LEFT] R=1600 μ s L=1400 μ s (7 s)
- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [SOFT TURN RIGHT] R=1550 μ s L=1700 μ s (7 s)

- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [SOFT TURN LEFT] R=1700 μ s L=1550 μ s (7 s)
- [NEUTRAL] R=1500 μ s L=1500 μ s (6 s)
- [WINCH] REEL IN fast freq=2000 Hz (9 s)
- [WINCH] REEL OUT fast freq=2000 Hz (9 s)

✓ **PASS** - Visual demo completed successfully *Press Enter to continue...* **PASSED**

tests/test_comprehensive.py::test_full_demo_all_layers

=====

[8/8] FULL DEMO — ROV (ARMED): LIGHTS, GRIPPER, THRUSTERS

===== ⚠ *CLEAR THE WATER around the ROV — it will be ARMED and its thrusters, lights, and gripper will move. Press Enter to start the full demo (Ctrl+C to abort)...*

```
##### #
ARMING ROV
##### Sent
mode change to MANUAL Sent arm command ✓ PASS - ROV armed and confirmed via
HEARTBEAT
```

```
##### #
ROV LIGHTS — off to brightest and back (button hold)
#####
holding LIGHTS BRIGHTER (button 13) for 6 s Sent MANUAL_CONTROL x=0 y=0 z=500
r=0 buttons=0 holding LIGHTS DIMMER (button 14) for 6 s Sent MANUAL_CONTROL x=0
y=0 z=500 r=0 buttons=0 ✓ PASS - Lights held brighter then dimmer (off -> brightest -> off)
```

```
##### #
ROV GRIPPER — full open then full close (button hold)
#####
holding GRIPPER OPEN (button 2) for 4 s Sent MANUAL_CONTROL x=0 y=0 z=500 r=0
buttons=0 holding GRIPPER CLOSE (button 1) for 4 s Sent MANUAL_CONTROL x=0 y=0
z=500 r=0 buttons=0 ✓ PASS - Gripper moved all the way open and all the way closed
```



```
##### #
ROV      THRUSTERS      —      all      8      directions
#####
```

- **FORWARD** Sent MANUAL_CONTROL x=400 y=0 z=500 r=0 buttons=0
SERVO_OUTPUT_RAW after FORWARD: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0
- **BACKWARD** Sent MANUAL_CONTROL x=-400 y=0 z=500 r=0 buttons=0
SERVO_OUTPUT_RAW after BACKWARD: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0
- **STRAFE RIGHT** Sent MANUAL_CONTROL x=0 y=400 z=500 r=0 buttons=0
SERVO_OUTPUT_RAW after STRAFE RIGHT: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0
- **STRAFE LEFT** Sent MANUAL_CONTROL x=0 y=-400 z=500 r=0 buttons=0
SERVO_OUTPUT_RAW after STRAFE LEFT: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0
- **YAW RIGHT** Sent MANUAL_CONTROL x=0 y=0 z=500 r=400 buttons=0
SERVO_OUTPUT_RAW after YAW RIGHT: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0
- **YAW LEFT** Sent MANUAL_CONTROL x=0 y=0 z=500 r=-400 buttons=0
SERVO_OUTPUT_RAW after YAW LEFT: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0
- **ASCEND** Sent MANUAL_CONTROL x=0 y=0 z=700 r=0 buttons=0
SERVO_OUTPUT_RAW after ASCEND: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0

- **DESCEND** Sent MANUAL_CONTROL x=0 y=0 z=300 r=0 buttons=0
SERVO_OUTPUT_RAW after DESCEND: {1: 1488, 2: 1494, 3: 1506, 4: 1500, 5: 1493, 6: 1507, 7: 1493, 8: 1507} Sent MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0

✓ **PASS** - ROV thrusters exercised in all 8 directions

TEARDOWN

Sent
MANUAL_CONTROL x=0 y=0 z=500 r=0 buttons=0 Sent disarm command ✓ **ROV**
disarmed

SUMMARY

✓
PASS - ROV armed, lights/gripper/thrusters all exercised

Press Enter to continue... **PASSED**

=====
TEST COMPLETE
=====

13. Mission Planning and Autonomous Execution via QGroundControl

Following the successful implementation of the MAVLink GPS-injection bridge, the ROV's Extended Kalman Filter (EKF) maintains a stable 3D localization fix. This critical capability allows the system to utilize the native mission planning tools within QGC to execute fully autonomous operations, such as the project's target 20×20 meter polygonal area scan.

The following outlines the procedural workflow for defining, uploading, and executing autonomous missions using the QGC interface.

13.1 Defining an Autonomous Waypoint Mission (Auto Mode)

To instruct the ROV to autonomously survey a specific underwater area, the operator utilizes QGC's "Plan" interface to generate a structured waypoint grid.

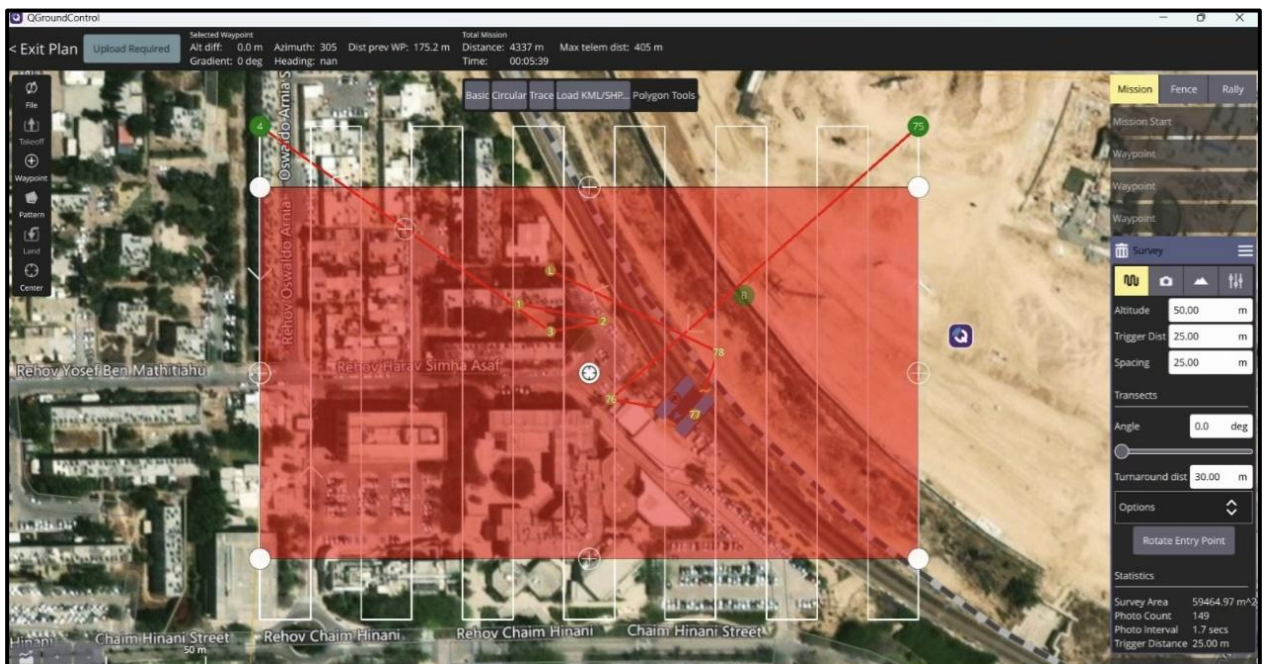


Figure B.14: Plan polygon dive

1. **Accessing the Plan View:** The operator navigates to the "Plan" section in QGC, which opens the global satellite map interface.
2. **Creating a Survey Pattern:** Instead of placing individual waypoints manually, the operator selects the Pattern tool. This allows the operator to draw a polygon directly on the map, defining the boundaries of the operational area (e.g., the targeted 20×20 meter perimeter).

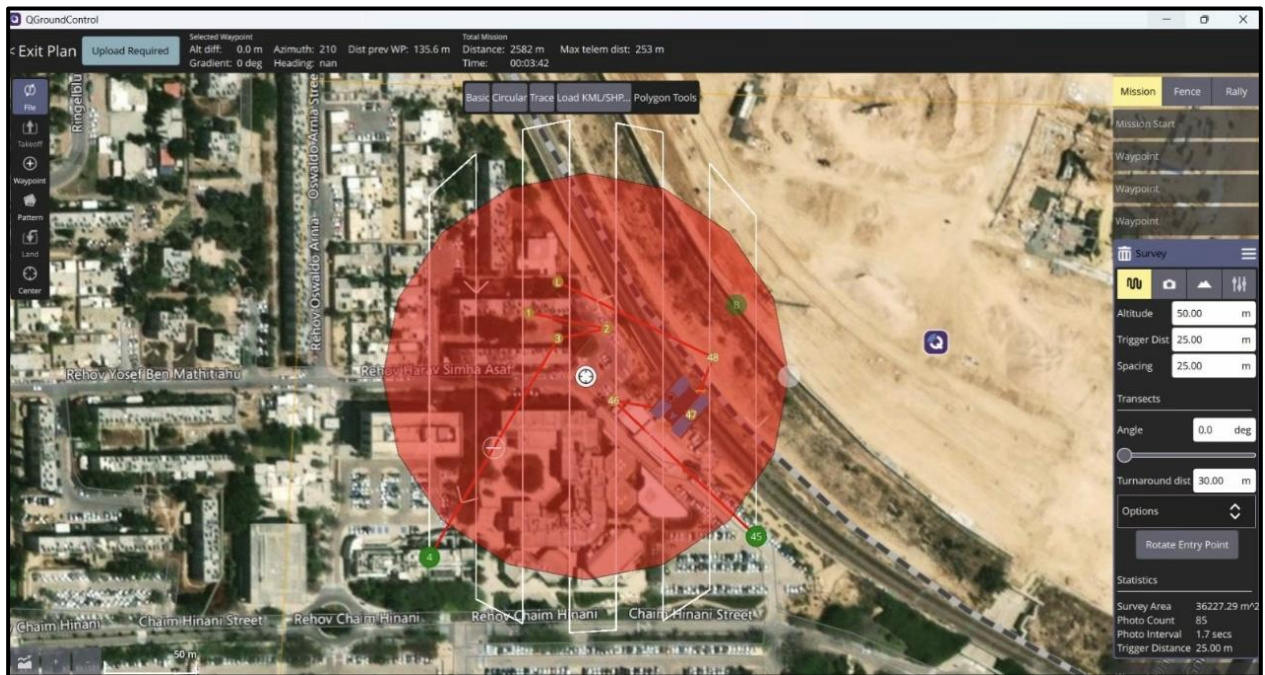


Figure B.15: Plan cycle dive

3. Once the polygon is defined, QGC automatically generates a sweeping grid pattern (transects) to cover the area. The operator must then adjust mission-specific parameters in the right-hand control panel:
 - **Altitude/Depth:** Setting the specific negative altitude (depth) the ROV should maintain during the survey.
 - **Transect Spacing:** Adjusting the distance between the parallel sweeping lines to ensure proper camera/sensor overlap.
 - **Turnarounds:** Configuring how the ROV behaves at the end of each line (e.g., stopping and rotating, or flying through the turn).

13.2 Mission Synchronization and Upload

Defining the mission in QGC only creates a local file on the operator's laptop. To execute the mission, the data must be transmitted over the RF link, through the USV relay, and into the ROV's ArduSub flight controller.

Must press Upload Required and then Upload.

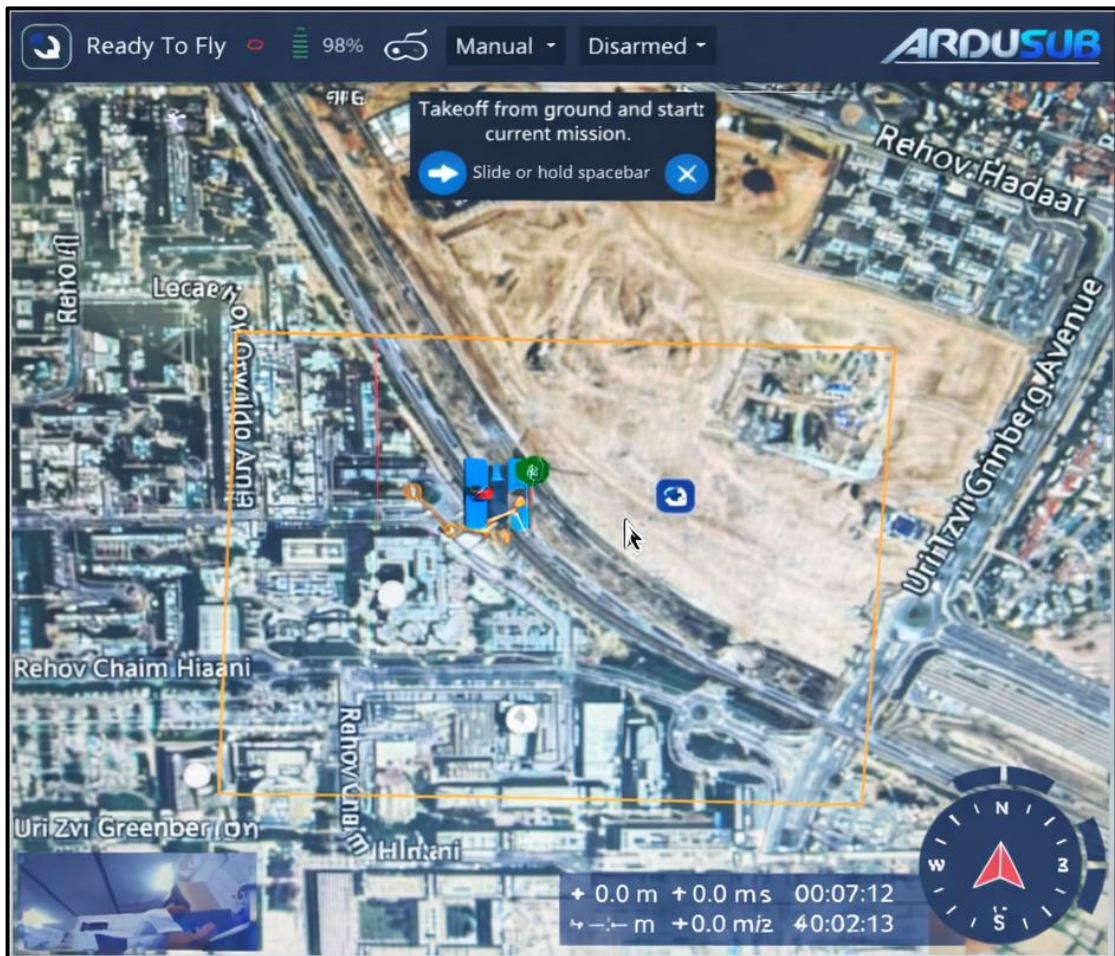


Figure B.16: Auto mode screen

- **The Upload Process:** The operator clicks the Upload button in the top right corner of the Plan screen. QGC translates the visual grid into a sequence of MAVLink MISSION_ITEM messages and transmits them to the ROV.
- **Verification:** The system confirms the upload is complete when the "Upload Required" warning disappears. The ROV's onboard memory now holds the exact geographic coordinates, depth requirements, and sequential order of the mission.

13.3 Activating 'Auto' and 'Guided' Modes

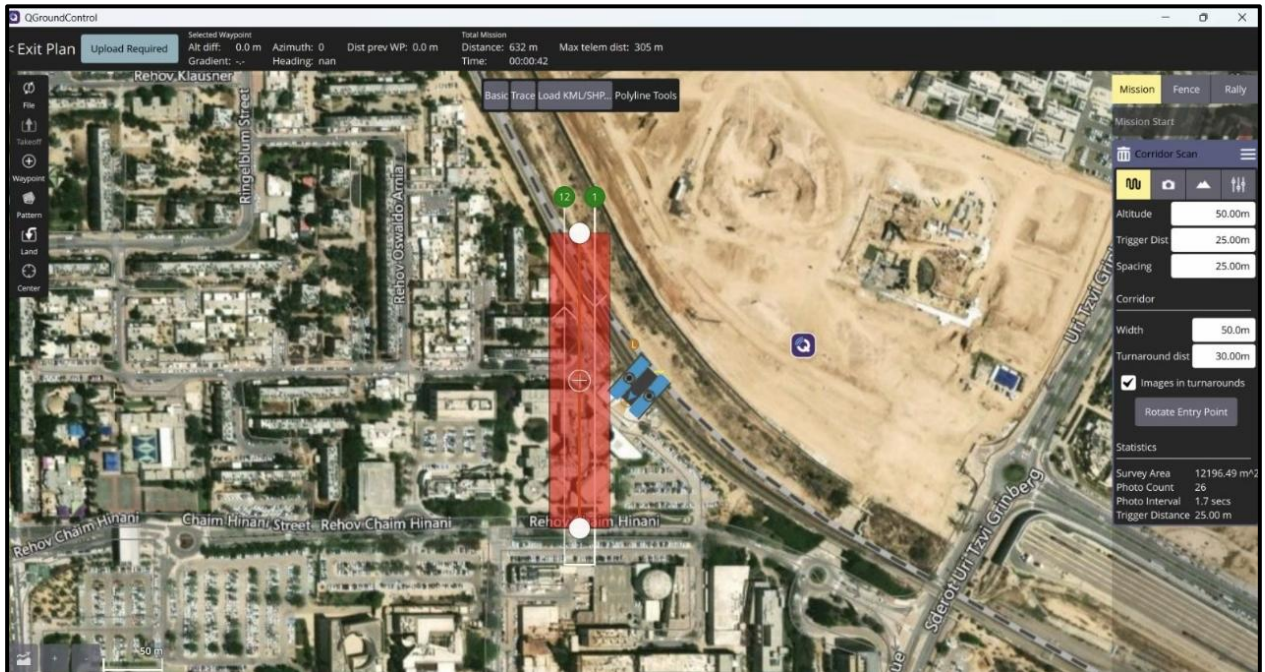


Figure B.17: Plan grid dive

Once the mission is uploaded and the ROV is actively receiving the injected GPS coordinates from the USV, the operator can initiate autonomous flight.

- **Entering AUTO Mode:** The operator arms the vehicle and selects Auto from the flight mode dropdown menu (or via a pre-mapped joystick button). The ArduSub flight controller immediately takes over propulsion control, navigating the ROV to the first waypoint and proceeding sequentially through the uploaded grid.
- **Entering GUIDED Mode:** If a pre-planned grid is not required, the operator can utilize **Guided** mode for dynamic, on-the-fly autonomy. While in the "Fly" view, the operator simply clicks on a specific location on the QGC map and selects "Go to Location". QGC sends a single target coordinate to the ROV, which autonomously calculates the trajectory and drives to that specific point without requiring a full mission upload.

14. Troubleshooting

What you see / hear	What to do
No second sound from the ROV after 2 minutes	Check that all cables are plugged in firmly. Disconnect the ROV battery, wait 10 seconds, then reconnect it and wait 2 minutes again, or check the ROV battery
Terminal says “connected: False”	Type this and press Enter, then try the test again: sudo pigpiod
A thruster does not beep when powered on	Turn off the power supply output immediately. Check that the wire from the Pi to the ESC is firmly connected, then turn the output back on
A power supply screen shows “ALARM” and shuts off	Press the Output button to turn it off completely. Increase the OVP (upper voltage limit) setting on that power supply by 2–3 V, then turn it back on
Tension reading on screen jumps to zero sometimes	This is a known, minor issue. You can continue — it does not stop the system from working correctly
Encoder number does not change when you move the cable by hand	Turn everything off. Check that all 4 small wires on the encoder are firmly plugged into the correct pins, then turn the system back on and try again
Winch motor does not move at all	Turn off the winch power supply. Check that all 3 wires from the Pi to the winch motor are firmly connected, then turn the power back on
No GPS signal / position shown	Move the GPS antenna to a spot with a clear, open view of the sky (not indoors, not under cover). Wait a few minutes

The test program shows a red error message instead of PASSED/FAILED	Make sure you moved into the project folder first: <pre>cd /home/lar/USV</pre> and that you ran the test with python3 (not just pytest): <pre>python3 -m pytest tests/test_comprehensive.py -v -s</pre>
Cannot ping the antennas (10.0.0.10 / 10.0.0.11) or the ROV, even though the link "worked before"	Check the USV Pi's network. Type this and press Enter: <pre>ip a</pre> Confirm the br0 line shows inet 192.168.2.3, and look for an interface with a 10.0.0.x or expected address. If an interface shows DOWN or NO-CARRIER, that cable/link is the problem
Want to see all interfaces and their IP addresses (older style)	Type this and press Enter: <pre>ifconfig</pre> Check that eth0, eth1 (the FXTI), and br0 are listed and have the expected addresses. A missing interface means its cable or USB device is not connected
Cannot reach the ROV (192.168.2.2) but the antennas are fine	Check the routing and which devices answer. Type these and press Enter: <pre>ip route arp -n</pre> If 192.168.2.2 shows "(incomplete)", nothing is answering — the ROV's link (FXTI/tether) is down or BlueOS has not finished booting. Confirm the FXTI Power and Link lights are on, then wait 2–3 minutes
The FXTI (USB-Ethernet to the ROV) does not appear as an interface	Type this and press Enter: <pre>lsusb</pre> Look for "Realtek RTL8152" (the FXTI). If it is missing, re-seat the FXTI's USB cable in the Pi's blue port and check its power

Table B.13: Common problems and what to do about them

Click here to see more <https://bluerobotics.com/learn/bluerov2-operation/>

15. Safety Summary

The following safety points are consolidated from throughout this manual and must be observed by all operators.

- Never power the cable angle encoder or HX711 amplifier from 5 V while their signal lines are wired directly to Raspberry Pi GPIO pins; use 3.3 V or an appropriate level shifter.
- Always set bench power supply current limits and over-voltage protection (OVP) with headroom before connecting any thruster or motor; fast motor reversal causes regenerative spikes that will trip a tight OVP.
- Never run T200 thrusters dry for more than a few minutes; secure them before any test, with the propeller area clear of hands, cables, and loose objects.
- Never operate the winch motor with the tether at the end of its travel; always confirm free travel before powering the winch.
- Never modify or interfere with the ROV's own Raspberry Pi 4 (BlueOS/ArduSub) as part of USV development — it is a separate unit from the USV Pi.
- Never discharge the ROV 4S battery below 12.8 V; monitor voltage during operation, and do not operate below 50% charge.
- Always keep the ROV tether clear of boat propellers, sharp objects, and other vessels during deployment and recovery.
- Always confirm at least 3 GB of free disk space on the ROV before a session.
- Always perform a full hardware test bench run before any pool or open-water trial.

Note: This manual reflects the system configuration and bring-up status as of the current project phase. Update it as additional subsystems (e.g. finalized GPS injection, full ROS 2 node graph, in-water trials) are completed and validated.

9.4 Links Table

Name	Link
Blue Robotics website	https://bluerobotics.com
Antenna website	http://10.0.0.X
BlueOS – ROV operating system	192.168.2.2
Project GitHub link	https://github.com/Maayan-Sarig/USV-Project

Table 4: Links Table

9.5 Recommendation Sheet for Grading a Summary Report

המלצת ציון לדו"ח מסכם

אם יש צורך, לכל סטודנט/ית בנפרד

מספר הפרויקט: _____ - P-20, שם הפרויקט:

שם המנחה:

ת.ז.:

שם הסטודנט/ית:

5-מצוין	4 - ט"מ	3 - טוב	2 - בינוני	1 - חלש	קריטריון	%
					הגדרת המטרה - האם מטרת הפרויקט ברורה? המטרה צריכה לכלול גם את התרומה הצפויה מהשלמת הפרויקט.	9
					ארכיטקטורת הפתרון - האם הובהר מדוע הארכיטקטורה המוצעת לפתרון ההנדסי מתאימה לפתרון הבעיה? האם הייתה התייחסות לאלטרנטיבות?	10
					הצגת התוצאות - האם הוסברו המשמעויות של התוצאות, או המסקנות הנובעות מהן, האם רק הייתה הצגה עובדתית של תוצאות?	9
					מבנה – מצגת - האם יש מבנה הגיוני להצגת העבודה? מבוא, רקע ו-SoTA, מטרות, שיטה, פתרון, תוצאות, מסקנות.	9
					רמת קושי - כיצד הנך מעריך את הפרויקט בהשוואה לפרויקטים אחרים השנה או בשנים קודמות?	9
					איכות כתיבה / הצגה - נא להתייחס לאיכות הצגת העבודה: תחביר, מבנה וקשר הגיוני בין החלקים במהלך הצגת העבודה	9
					שורה תחתונה - האם מטרות הפרויקט הושגו? יש להתייחס למטרות כפי שהוגדרו בתחילה.	9

					9 הבנה - האם נראה שהסטודנט מבין את העבודה שנעשתה? האם עומק הדיון מספק?
					9 השקעה - האם ניכר שהסטודנט השקיע בעבודה בפרויקט ו/או בהכנת ההצגה של העבודה? האם נראה שהעבודה נעשתה בחופזה, כדי לצאת ידי חובה?
					9 דירוג - כיצד הנך מעריך את איכות הפרויקט ביחס לכלל הפרויקטים שראית?
					9 המאפיינים העיקריים בפרויקט - יש לסמן את כמות המאפיינים שהושגו (ולו באופן חלקי). מאפיין אחד בעמודה 1 וכן הלאה... המאפיינים: * מימוש/יצור מערכת, * אנליזה מתמטית, * תכנון אלגוריתמי, * אינטגרציה עם מערכת קיימת, * חקר ביצועים.

אם יש כוונה לפרסם/ יפורסם מאמר, שם כתב העת ומועד משוער להגשה:

ציין אם יש כוונה לשקול המלצה כפרויקט מצטיין:

הערות נוספות :